

Grado en Ingeniería Informática
2024-2025

Trabajo Fin de Grado

“Desarrollo de un dataset para el cálculo de distancias entre núcleos urbanos y servicios de ámbito comarcal con titularidad pública en la Comunidad de Madrid”

Raúl García Romero

Tutor/es

Javier García Guzmán

Leganés, 2025



Esta obra se encuentra sujeta a la licencia Creative Commons
Reconocimiento – No Comercial – Sin Obra Derivada

RESUMEN

El siguiente Trabajo de Fin de Grado está orientado a el cálculo de distintos indicadores de disponibilidad de distintos servicios públicos para los distintos municipios de la Comunidad de Madrid, a partir de la obtención de la distancia media entre cada núcleo urbano y un conjunto con cada uno de los siguientes servicios públicos: Hospitales, juzgados, centros de salud mental y parques de bomberos.

Inicialmente, los datos utilizados para llevar a cabo esta práctica son distintos datasets georreferenciados, es decir, un conjunto de instancias de datos cada una ligada a un punto geográfico por medio de coordenadas. Estos datos serán preprocesados, procesados y utilizados en programas de Python y QGIS para obtener distintos conjuntos de datos que den la información necesaria.

ÍNDICE DE CONTENIDOS

1.	INTRODUCCIÓN.....	1
1.1.	Motivación del Trabajo.....	1
1.2.	Área de Trabajo	1
1.3.	Problema por resolver.....	3
1.4.	Objetivos.....	3
2.	ESTADO DE LA CUESTIÓN	5
2.1.	Trabajos previos y documentación existente	5
2.2.	Concepto de cohesión territorial y justicia/equidad espacial.....	6
2.3.	Estudios sobre disparidades territoriales en el acceso a servicios públicos	6
2.4.	Marco regulatorio	7
2.5.	Noticias sobre disponibilidad de servicios públicos en Madrid	8
3.	PLAN DE TRABAJO	9
3.1.	Obtención de los datos.....	9
3.2.	Preprocesamiento de datos	10
3.3.	Procesamiento de datos	10
3.3.1.	Procesamiento de los datos mediante QGIS.....	11
3.3.2.	Procesamiento de los datos mediante Python.....	11
3.4.	Cálculo de distancias utilizando la API de Google Maps y creación del dataset resultante.....	12
3.5.	Descripción del dataset.....	13
3.6.	Creación del visor GIS	13
4.	METODOLOGÍA PARA LA CONSTRUCCIÓN DEL DATASET.....	14
4.1.	Obtención de los datos iniciales	14
4.2.	Preprocesado de datos	16
4.3.	Procesado de datos	16
	Filtrado de diseminados y núcleos con menos de 10 habitantes	18
	Obtención de los centroides.....	20
	Adición de atributos	25
4.4.	Selección de vecinos más cercanos a través de la librería de QGIS para Python	28
	Métodos <code>closest_destinations_cords()</code> y <code>closest_destinations_features()</code>	28
4.4.	Consulta a la Routes API de Google	31
	Métodos para la realización de consultas	31
4.5.	Depurado de los resultados.....	35

4.6. Unión con el dataset en formato Geopackage	36
Capa de núcleos poblacionales	37
Capa de municipios	38
5. DESCRIPCIÓN DEL DATASET OBTENIDO	40
6. ESPECIFICACIÓN DE REQUISITOS DE LA APLICACIÓN GIS PARA VISUALIZAR LOS DATOS.....	44
7. DISEÑO	47
8. RESULTADOS	53
9. CONCLUSIONES Y FUTUROS TRABAJOS.....	54
Referencias	55

ÍNDICE DE FIGURAS

Ilustración 1 Distribución espacial de estado socioeconómico y distancia media a instalaciones deportivas en Madrid	6
Ilustración 2 Ilustración del artículo de Roca y Moreno-Jimenez.....	7
Ilustración 4 Vista previa de la herramienta QGIS.....	¡Error! Marcador no definido.
Ilustración 5 Vista previa del código usando la API de QGIS.....	¡Error! Marcador no definido.
Ilustración 6 Ejemplo de consulta a Routes API	12
Ilustración 7 Ejemplo de respuesta de Routes API.....	13
Ilustración 3 Captura de pantalla de la página web NOMECALLES	15
Ilustración 8 Proceso de filtrado de núcleos poblacionales.....	20
Ilustración 9 Ajuste de centroide a carretera	23
Ilustración 10 Ajuste a parada de bus más cercana	¡Error! Marcador no definido.
Ilustración 11 Tabla de hospitales de grupo 2 y 3.....	¡Error! Marcador no definido.
Ilustración 12 Núcleos y hospitales clasificados por su DAS	26
Ilustración 13 División por partido judicial.....	27
Ilustración 14 Fragmento del output de closest_dest_cords()	30
Ilustración 15 Fragmento del output de closest_dest_features()	31
Ilustración 16 Ejemplo de JSON despues de penalizaciones	36

ÍNDICE DE CÓDIGO

Código 1 Método <code>closest_destinations_features()</code>	29
Código 2 Método <code>closest_destinations_cords()</code>	30
Código 3 Método <code>request_routes_v2_async</code>	33

LISTA DE ABREVIATURAS

QGIS	Quantum Geographic Information System
DAS	Dirección Asistencial de Salud
INE	Instituto Nacional de Estadística

GLOSARIO

Término	Descripción
Núcleo de población	Conjunto de al menos diez edificaciones y/o cincuenta habitantes con calles, plazas u otras vías urbanas. [1]
Diseminado	Edificaciones o viviendas que no pueden ser incluidas en el concepto de núcleo de población. [2]

1. INTRODUCCIÓN

1.1. Motivación del Trabajo

Asegurar un abastecimiento adecuado de servicios públicos de primera necesidad es, o debería ser, una de las mayores prioridades de las administraciones públicas, ya que la calidad de vida de una gran parte de la ciudadanía depende de un fácil acceso a estos servicios. El acceso equitativo a servicios públicos esenciales constituye uno de los pilares fundamentales del Estado de Bienestar y un indicador clave del desarrollo territorial equilibrado.

En un mundo industrializado y con la infraestructura necesaria para conectar todos los núcleos urbanos de un país, es cada vez más fácil y necesario determinar qué servicios públicos brindar y dónde se establece la construcción de la infraestructura destinada a brindar dicho servicio, dejando el menor número de áreas incomunicadas posible. Sin embargo, la complejidad de los sistemas territoriales actuales requiere de metodologías sistemáticas y herramientas de análisis espacial que permitan evaluar objetivamente la accesibilidad real a estos servicios.

1.2. Área de Trabajo

El área de trabajo de este estudio son los 179 municipios que componen la Comunidad de Madrid, así como los núcleos poblacionales que lo forman y las distintas infraestructuras donde se brindan servicios públicos esenciales. Dichos municipios presentan diferencias notables en termino de desarrollo de infraestructuras, densidad de red de carreteras y frecuencia de transporte público.

La Comunidad de Madrid abarca una superficie de 8.028 km² [3] y cuenta con una población de más de 7 millones de habitantes (INE, 2024) [4], lo que la convierte en una de las regiones más densamente pobladas de Europa. No obstante, esta densidad presenta una distribución extremadamente heterogénea: mientras que el municipio de Madrid concentra más del 48,8% de la población regional, existen 52 municipios con menos de 1.000 habitantes que ocupan extensas áreas rurales.

Esta disparidad territorial genera problemas en términos de accesibilidad a servicios públicos. Por un lado, las áreas metropolitanas centrales acaparan la mayoría de los servicios especializados, mientras que las zonas periféricas y rurales presentan una baja cobertura de servicios básicos.

Ámbito territorial:

- **Núcleos poblacionales:** Se analizan 468 núcleos de población con más de 10 habitantes. En la comunidad existen más de 1.208 áreas consideradas como núcleos poblacionales, pero excluyendo diseminados y núcleos con población inferior al umbral establecido el conjunto de núcleos a considerar se reduce a prácticamente un tercio.

- **Delimitación municipal:** El conjunto de los 179 municipios que forman la Comunidad de Madrid
- **Otras delimitaciones administrativas:** Existen distintas divisiones de territorio que pueden ser relevantes en el estudio. Algunos ejemplos son los partidos judiciales o las Direcciones Asistenciales de Salud.
- **Red de carreteras:** Carreteras principales y secundarias que interconectan los núcleos poblacionales.

Servicios públicos:

- **Hospitales de grupo 2 y 3:** Hospitales de mayor complejidad, comúnmente llamados “Hospitales de referencia”, los cuales cuentan con la mayor cantidad de especialidades.
- **Centros de Salud Mental:** Incluye unidades y centros de Salud Mental y centros psiquiátricos de toda la comunidad.
- **Juzgados:** Audiencias Nacional y Provincial, oficinas de Registro Civil, Centros de Primera Instancia, Fiscalías y otros tipos de juzgados de la comunidad.
- **Parques de bomberos:** Conjunto de los 36 parques de bomberos del territorio

Ámbito tecnológico: Durante el desarrollo del trabajo utilizamos distintas herramientas de software:

- **QGIS:** Primera etapa de preprocesamiento de los datos obtenidos y visualización de la información
- **Automatización de procesos:** Desarrollo de distintos scripts en Python para manejo y conversión de distintos archivos, así como el uso de la librería `qgis.core`, que permite operar sobre capas de QGIS en Python, permitiendo operaciones más complejas.
- **Uso de APIs:** Llamadas a la Routes API de Google para el cálculo de distancias reales por carretera y en distintos medios de transporte
- **Frontend:** Desarrollo de una interfaz básica para la visualización del dataset resultante.
- **Backend:** Desarrollo de un servidor en Flask con Python para leer los datasets y proporcionar una API web sencilla que permita acceder a los datos fácilmente sin necesidad de pasar por QGIS.
- **Formalización de dataset:** Uso del estándar DCAT para formalizar los metadatos de los datasets obtenidos.

La información sobre este dominio de trabajo la tratamos en forma de datasets georreferenciados que nos permiten operar sobre ellos, obtener información adicional y permite la visualización de dichos datos en el mapa mediante aplicaciones como QGIS, ArcGIS o distintas librerías y aplicaciones como la librería de QGIS para Python y OpenLayers.

1.3. Problema por resolver

Un problema que genera bastante preocupación desde hace muchos años es la falta de servicios públicos en zonas rurales o alejadas de centros de provincia y ciudades importantes. A esto se le suma la falta de datos empíricos y demostrables que validen y cuantifiquen el grado de accesibilidad de los servicios públicos y cuáles son los servicios públicos más necesarios para reforzar en cada zona.

Esta problemática no se ha resuelto con el paso de los años debido a varios factores. La primera es que la población en el centro de Madrid no ha hecho más que aumentar, haciendo que las administraciones públicas se centren en seguir abasteciendo el centro de la ciudad con servicios, dejando a la periferia cada vez más descuidada. En segundo lugar, está la falta de inversión en servicios públicos, que obviamente afecta más duramente a los pueblos de la periferia.

Existe además una falta de indicadores objetivos y comparables de disponibilidad y accesibilidad a servicios públicos por territorio. Algunos estudios como el de Roca y Moreno-Jimenez [5] tienen en cuenta otros factores adicionales como la renta per cápita por territorio, el cuál, aunque no se contemple en este trabajo, sería sin duda un aspecto a considerar a la hora de calcular un “índice de disponibilidad de servicios”. A simple vista parece imposible evaluar objetivamente qué municipio carece de qué servicio público, pero se puede obtener un índice a partir de distintos datos, el caso es determinar cuáles y hacer un estudio exhaustivo.

Por lo tanto, el problema que pretendemos abordar es el del desarrollo de una base de datos en la que se pueda cuantificar la disponibilidad de servicios en todas las áreas de la Comunidad de Madrid en términos de distancias de viaje y duración del trayecto, tanto en vehículo personal como en transporte público.

1.4. Objetivos

Este trabajo tiene como meta la obtención de información y la posterior creación de una base de datos sobre la disponibilidad de distintos servicios públicos en los diferentes municipios de la Comunidad de Madrid, procurando que la metodología sea lo más replicable posible.

El objetivo general es desarrollar un dataset georreferenciado que contenga una gran cantidad de indicadores de proximidad, accesibilidad o disponibilidad de servicios públicos esenciales tanto a nivel de núcleo poblacional como a nivel de municipio.

El objetivo general está claro, pero vamos a establecer los **objetivos específicos** del trabajo:

1. **Obtención de los datos:** Descarga de los datasets iniciales de fuentes fiables
2. **Preprocesamiento de los datos:** Convertir los datasets a un formato con el que se puedan realizar operaciones sobre ellos, sobre todo para ser leídos por scripts en Python

3. **Procesamiento de los datasets:** Operar sobre los datasets ya preprocesados para obtener, para cada núcleo urbano, un listado de los servicios públicos cercanos con los que calcularemos la distancia, ya que nos es irrelevante la distancia entre un núcleo poblacional y un servicio en extremos opuestos de la comunidad si hay servicios más cercanos.
4. **Automatización de las llamadas a Routes API y obtención de las distancias y duraciones:** Google Maps Platform cobra por cada llamada a la Routes API, por lo que es necesario un plan de requests que tome todas las respuestas, las agrupe y, en caso de algún error por formato realice nuevas requests sólo para los datos que faltan.
5. **Formalización del dataset y creación de herramientas de visualización:** El dataset obtenido de por sí ya debería ser información suficiente, pero también urge presentar de forma accesible los resultados obtenidos, ofreciendo representaciones visuales

Software utilizado:

- **QGIS**, software de gestión de bases de datos con representación geográfica de la información.
- **Python, HTML y Javascript:** Todos los scripts de automatización se harán en Python, mientras que el visor web se desarrollará con HTML y Javascript.
 - **Librerías más utilizadas:** pandas, numPy, qgis.core, Flask.
- **Routes API de Google Maps Platform:** API proporcionada por Google que nos permitirá el cálculo de distancias de forma sencilla.

2. ESTADO DE LA CUESTIÓN

2.1. Trabajos previos y documentación existente

El grado de facilidad de acceso y cercanía a servicios públicos depende de una gran cantidad de factores: tráfico, listas de espera, cercanía, conexión por transporte público, accidentes naturales, obras en la zona del servicio público o cambios en la legislación. No es fácil encontrar un valor o una calificación que indique al 100% el grado de facilidad que tienen los habitantes de cada municipio para acceder a cada uno de los distintos servicios públicos de ámbito comarcal. Dicho esto, consideramos la cercanía por carretera y por transporte público un primer enfoque que nos permite acercarnos a esa calificación que mencionamos.

La accesibilidad a servicios públicos es un factor fundamental a tener en cuenta en la planificación territorial y las políticas públicas.

En *Accessibility, equity and health care: review and research directions for transport geographers* [6] y en *The Concept of Access: Definition and Relationship to Consumer Satisfaction* [7] se coincide en que el acceso a servicios de atención sanitaria se ve afectada por 5 factores:

- Asequibilidad: Coste del servicio
- Aceptabilidad: Satisfacción general de los usuarios
- Disponibilidad: Adecuación del servicio a todo tipo de usuarios
- Accesibilidad geográfica: Impedancia de viaje entre paciente y proveedor.
- Comodidad o acomodación: Cómo de apropiado y conveniente es el servicio.

Entre estos factores podemos identificar que la accesibilidad geográfica es exactamente el punto que se quiere tratar en este estudio.

Wang y Luo (2005) establecieron los fundamentos metodológicos para medir la accesibilidad espacial a servicios de salud, desarrollando el método de áreas de captación flotantes (Two-Step Floating Catchment Area), que considera tanto la oferta de servicios como la demanda poblacional.

Aunque no exista una gran cantidad de trabajos previos sobre el tema que tratamos de forma específica, pero sí podemos ver que se han sentado bases y hay antecedentes de distintos estudios analizando el problema de la falta de cobertura espacial de servicios públicos esenciales, sobre todo en el ámbito de la sanidad debido a su importancia. En español existe un menor número de estudios sobre el tema, pero sí se dispone de documentos oficiales, aplicaciones y datasets que, aunque no hablen ni traten las distancias entre servicios públicos y municipios o núcleos poblacionales, aportan datos georreferenciados con información de estos puntos.

Los primeros recursos sobre los que nos apoyamos para realizar este trabajo son la plataforma NOME CALLES [8], con históricos de datos georreferenciados aportados por la Comunidad de Madrid, así como el sitio web del Consorcio de Transportes de

Madrid [9], que está dotado de un buscador en el que se pueden encontrar datasets relacionados con transporte público de la comunidad.

2.2. Concepto de cohesión territorial y justicia/equidad espacial

La cohesión territorial es un concepto que fue introducido durante la época de la Europa de posguerra, con la intención de describir la necesidad de llevar servicios esenciales y conectar zonas poco pobladas. En 2009, Katja Mirwaldt, Irene McMaster y John Bachtler presentan un paper que relata la problemática dentro de la Unión Europea a la hora de definir bien este término y permitir así un plan conjunto dentro de la UE [10]. Por lo tanto “cohesión territorial” aún se usa como término puente para representar la necesidad de evitar disparidades socioeconómicas y desequilibrios en nivel de accesibilidad a servicios esenciales.

Respecto al concepto de justicia espacial o equidad espacial se han realizado varios trabajos abordándolo:

- Taleai et al. (2014) [11]: Define la equidad espacial como el grado en el que los servicios se distribuyen espacialmente de manera igualitaria sobre diferentes áreas correspondientes a la variación espacial de la “necesidad” de dichos servicios. Además establecen el índice Integrated Spatial Equity Evaluation (ISEE), que se centra en medir el balance entre oferta y demanda de servicios por territorio.
- Jian et al. (2019) [12] rescatan este término para hablar de la necesidad de creación de espacios abiertos en zonas más necesitadas de ellos

2.3. Estudios sobre disparidades territoriales en el acceso a servicios públicos

- Luis Cereijo et al.(2019) [13]: Acceso y disponibilidad de instalaciones deportivas en Madrid. Estudio que refleja un hecho cada vez más notable en la comunidad, la disparidad entre habitantes del centro y habitantes de la periferia

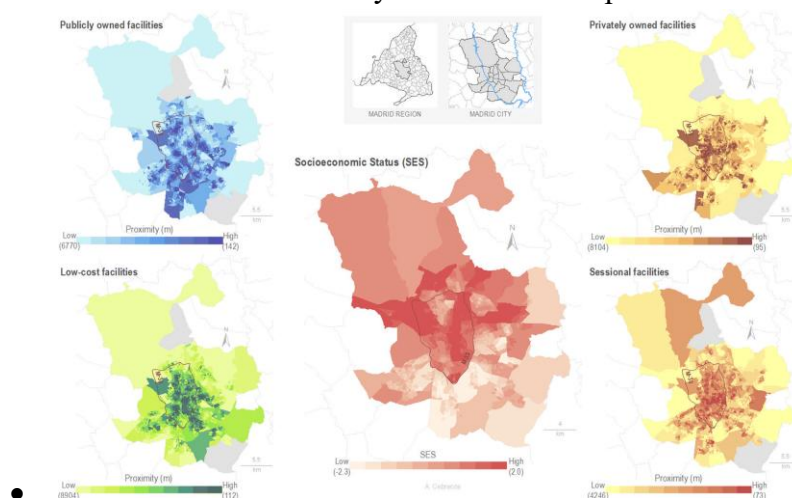


Ilustración 1 Distribución espacial de estado socioeconómico y distancia media a instalaciones deportivas en Madrid

- Andrés Roca y Moreno-Jimenez Antonio en su trabajo “Accesibilidad de los jóvenes a la red de transporte público en Alcobendas (Madrid) por motivo de ocio nocturno” [5] hacen un estudio del grado de disponibilidad de transporte público en la zona de Alcobendas, teniendo en cuenta puntos clave como nivel de renta medio de la zona, tiempo de desplazamiento a bocas de Metro o paradas de bus y accesibilidad a otras zonas de ocio. En general, muestra la gran cantidad de factores que forman parte de lo que entendemos como “disponibilidad” de un servicio público.

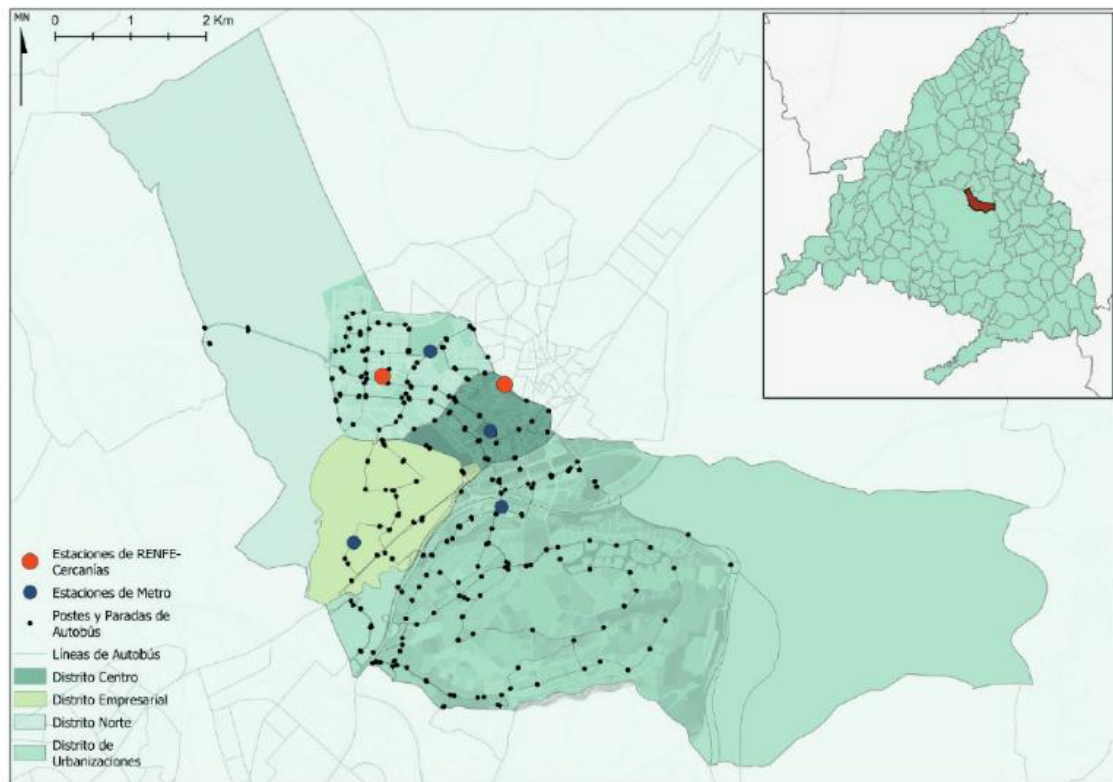


Ilustración 2 Ilustración del artículo de Roca y Moreno-Jimenez

2.4. Marco regulatorio

En la Directiva de la Unión Europea 2019/1024 [14] la Ley 37/2007 [15] y recoge la necesidad de las administraciones públicas de hacer públicos datasets o recursos electrónicos para uso en diversos campos como la investigación.

A causa de esto podemos encontrar a día de hoy una gran cantidad de datos publicada por organizaciones como el Instituto Nacional de Estadística, el Consorcio Regional de Transporte o el Instituto de Estadística en portales como nomecalles

En lo que respecta a la planificación de servicios públicos, ya mencionamos que la Unión Europea apuesta por un plan de cohesión territorial, pero como se puede ver en el Artículo 174 del Tratado de Funcionamiento de la Unión Europea [16], este plan no está nada definido y se trata como una sugerencia, mas que una ley: “A fin de promover un desarrollo armonioso del conjunto de la Unión, ésta desarrollará y proseguirá su acción encaminada a reforzar su cohesión económica, social y territorial. Entre las regiones afectadas se prestará especial atención a las zonas rurales...”.

2.5. Noticias sobre disponibilidad de servicios públicos en Madrid

En Madrid ha habido numerosos casos de fallos en abastecimiento de servicios públicos a personas necesitadas. En concreto, dos casos que están relacionados con el concepto de justicia espacial y cohesión territorial presentan situaciones que reiteran la necesidad de dotar de más instalaciones que brinden servicios públicos

- Falta de disponibilidad de centro de salud para consultas de pediatría en el municipio de San Martín de Valdeiglesias, obligando a sus vecinos a desplazarse 60km al centro de salud más cercano. [17]
- Una biblioteca por cada 101.000 madrileños [18]. Artículo que refleja la falta de disponibilidad de un servicio público del cual no se han construido instalaciones para satisfacer las necesidades de los madrileños.

3. PLAN DE TRABAJO

El desarrollo de este trabajo está diseñado para realizarse de forma completamente secuencial, exigiendo una buena planificación antes de pasar al siguiente paso, ya que cualquier error conlleva rehacer partes anteriores del proceso. Dado que en el punto 1.4. establecemos los objetivos principales del trabajo, los pasos deberían ser prácticamente los mismo:

1. **Obtención de los datos:** Recopilación de los datasets georreferenciados con información sobre núcleos poblacionales, delimitación de municipios, zonas de Dirección Asistencial de Salud o la posición de los servicios públicos sobre los que trata el trabajo
2. **Preprocesamiento de los datos:** Convertir los datasets a un formato con el que se puedan realizar operaciones sobre ellos, sobre todo para ser leídos por scripts en Python.
3. **Procesamiento de los datasets:** Operar sobre los datasets ya preprocesados para obtener, para cada núcleo urbano, un listado de los servicios públicos cercanos con los que calcularemos la distancia, ya que nos es irrelevante la distancia entre un núcleo poblacional y un servicio en extremos opuestos de la comunidad si hay servicios más cercanos.
4. **Obtención de métricas de accesibilidad:** Automatización de las llamadas a Routes API y obtención de las distancias y duraciones. Google Maps Platform cobra por cada llamada a la Routes API, por lo que es necesario un plan de requests que tome todas las respuestas, las agrupe y, en caso de algún error por formato realice nuevas requests sólo para los datos que faltan.
5. **Formalización del dataset:** La publicación de un dataset debe ir acompañado de una formalización de sus metadatos en DCAT, con información de palabras clave, autor, fecha de publicación y modificación y otros datos relevantes
6. **Creación de herramientas de visualización:**

3.1. Obtención de los datos

En esta etapa se identifican las potenciales fuentes de bases de datos con las que podemos empezar a operar. Estos datos tienen que ser legibles por QGIS para que podemos editarlos y visualizarlos.

El formato de los archivos puede ser tanto Shapefile, GeoPackage o GeoJSON.

Ciertos datos independientes o fáciles de interpretar sin necesidad de ser importados a QGIS pueden venir en formatos como hojas de Excel, CSV o simple .txt.

En relación a qué datos buscamos exactamente, como punto inicial tenemos esta lista:

- Núcleos poblacionales: Conjunto de los núcleos poblacionales con la suficiente densidad poblacional para ser relevantes en el estudio.

- Delimitación por municipios: Información de los 179 municipios de Madrid, se puede operar sin datos geográficos pero es preferible tener datos de las delimitaciones exactas de cada municipio para poder interactuar con otros datasets.
- Servicios públicos objeto de estudio: Datos georreferenciados de los servicios públicos de la Comunidad de Madrid con los que vamos a trabajar. Es importante tener información de la posición de cada servicio público para los siguientes pasos. Los servicios públicos de los cuales vamos a analizar su accesibilidad son:
 - Hospitales
 - Centros de salud mental
 - Parques de bomberos
 - Juzgados
- Red de carreteras de la Comunidad de Madrid: Trama de carreteras de la comunidad en formato de datos georreferenciados.

Ciertos datos que pueden ser utilizados sin necesidad de contener datos georreferenciados:

- Información de censo por núcleo poblacional o municipio: Normalmente en formato CSV por administraciones públicas.
- Datos sobre divisiones administrativas para distintos tipos de servicios públicos. Dependiendo del tipo de servicio, cada zona tiene un centro de referencia y distintas normas acerca de las poblaciones a las que da cobertura. Algunos ejemplos son los partidos judiciales en el caso de los juzgados o las Direcciones Asistenciales de Salud en caso de los hospitales.

3.2 Preprocesamiento de datos

Es necesario que exista consistencia en los atributos, por eso, una vez se tienen los datasets iniciales conviene asegurarse de que los tipos de datos son consistentes entre capas. Por ejemplo, un mismo tipo de dato, por ejemplo, número de habitantes, puede aparecer como número entero en una capa y en coma flotante en otra capa. Es muy útil para fases posteriores que estas diferencias se corrijan en la fase de preprocesado.

QGIS ofrece algunas herramientas que son útiles en el preprocesado de datos:

- Corregir geometrías: Algunos datasets pueden tener errores en los datos que dificulten el análisis en QGIS, con esta herramienta se evitan fallos durante el análisis.
- Reproyectar capa: Algunos datasets se publican con sistemas de coordenadas distintos, esta herramienta ayuda a reprojectarla con otro CRS.

3.3. Procesamiento de datos

El procesamiento de datos lo dividimos en dos partes, no necesariamente se hace una antes que la otra, pero cada una aporta distintas funcionalidades

QGIS permite operaciones generales sobre los datos, mientras que en Python se pueden crear scripts que ejecuten una gran cantidad de instrucciones de forma automática

3.3.1. Procesamiento de los datos mediante QGIS

QGIS ofrece una gran cantidad de herramientas que permiten manejar distintas capas de datos, operar sobre ellas, añadir, modificar o eliminar atributos en función su posición.

Dado que se van a realizar operaciones que enlazan instancias de datos de una capa con datos de otra capa distinta, es muy útil establecer identificadores únicos, así como capas intermedias con datos adicionales.

Herramientas de QGIS que pueden ser útiles en el proceso:

- Punto en superficie / Centroides: A partir de una capa de polígonos o delimitaciones, devuelve una capa de puntos representativos de cada delimitación conservando sus atributos.
- Unir atributos por localización: Tomando dos capas, copia la primera y le añade atributos de la segunda capa en función de si los elementos se intersecan, cortan, tocan o contienen, entre otras condiciones. Por ejemplo, se puede hacer que los elementos de la capa de núcleos poblacionales hereden el atributo identificador del municipio en el que se encuentran.

Otra funcionalidad que permite QGIS es el desplazamiento de puntos a mano en caso de necesitarlo.

3.3.2. Procesamiento de los datos mediante Python

La instalación de QGIS incluye también una librería de Python que permite el desarrollo de scripts en Python que accedan a capas de un proyecto QGIS y realicen operaciones con los datos.

Este paso se realiza en Python debido al excesivo backtracking que puede provocar cualquier fallo en el proceso o datos faltantes. Las herramientas de QGIS son útiles, pero algunos procesos requieren una línea de operaciones muy específicas que en QGIS serían extremadamente tediosas de realizar repetidamente y que en Python se podrían ejecutar en pocas líneas de código.

Durante este paso se deben realizar los siguientes procesos:

- Filtrado de núcleos poblacionales con pocos habitantes y diseminados
- Unión de atributos de distintas capas
- Creación de capas informativas
- Determinar para cada núcleo poblacional qué servicio público, dentro de cada categoría, se encuentra más cerca en distancia euclídea. Esto evita que tengamos que calcular posteriormente la distancia entre un núcleo poblacional y un servicio público que realmente no usan ni se debería usar.

3.4. Cálculo de distancias utilizando la API de Google Maps y creación del dataset resultante

Google Maps Platform ofrece acceso a Routes API [19] a usuarios registrados. Haciendo uso de esta API se pueden realizar consultas con un formato específico y obtener como resultado la distancia y tiempo exactos de un trayecto en coche a los conjuntos [origen, destino] que nosotros especifiquemos.

Debido a esto, es necesario tener un script que realice todas las consultas automáticamente, y organice las respuestas en un formato con el que se pueda operar en pasos posteriores.

```
curl -X POST -d '{
  "origin":{
    "location":{
      "latLng":{
        "latitude": 37.419734,
        "longitude": -122.0827784
      }
    }
  },
  "destination":{
    "location":{
      "latLng":{
        "latitude": 37.417670,
        "longitude": -122.079595
      }
    }
  },
  "travelMode": "DRIVE",
  "routingPreference": "TRAFFIC_AWARE",
  "computeAlternativeRoutes": false,
  "routeModifiers": {
    "avoidTolls": false,
    "avoidHighways": false,
    "avoidFerries": false
  },
  "languageCode": "en-US",
  "units": "METRIC"
}' \
-H 'Content-Type: application/json' -H 'X-Goog-API-Key: YOUR_API_KEY' \
-H 'X-Goog-FieldMask: routes.duration,routes.distanceMeters,routes.polyline.encodedPolyline' \
'https://routes.googleapis.com/directions/v2:computeRoutes'
```

Ilustración 3 Ejemplo de consulta a Routes API

```
{
  "routes": [
    {
      "distanceMeters": 772,
      "duration": "165s",
      "polyline": {
        "encodedPolyline": "ipkcFfichVnP@j@BLoFVwM{E?"
      }
    }
  ]
}
```

Ilustración 4 Ejemplo de respuesta de Routes API

Como se puede ver, la respuesta contiene información sobre la duración del viaje en segundos y la distancia en metros. En el ejemplo devuelve también un polyline que puede ser importado en QGIS, pero esto encarece el precio de la consulta y evitamos pedirlo.

Como último paso para la creación del dataset, este conjunto de resultados debe ser procesado y unido a los datasets existentes para quedarnos con un conjunto de datos georreferenciados que contenga la información original junto con todos los indicadores de accesibilidad obtenidos en el proceso.

3.5. Descripción del dataset

Para la correcta publicación del dataset es necesaria una formalización de sus metadatos en formato DCAT, el cual contiene una gran variedad de atributos ligados al dataset que ayudan a identificar, tanto para qué sirve, palabras clave, autor o fecha de creación.

3.6. Creación del visor GIS

Este dataset de por sí debería aportar suficiente información y podría ser cargado en QGIS sin problema, pero, inspirándonos en visores como el de nomecalles [8] queremos presentar los datos de forma que se pueda interactuar fácilmente con ellos, permitiendo ordenar por distintos atributos del dataset.

4. METODOLOGÍA PARA LA CONSTRUCCIÓN DEL DATASET

En este punto se explicará en más detalle los pasos realizados, así como los resultados obtenidos durante el proceso.

Cabe aclarar que aunque estas decisiones se presentan de forma lineal, algunas se toman en partes más tardías del desarrollo por motivo de backtracking. En estos casos, se aclarará el motivo de las decisiones o se dejará claro que se deben a etapas posteriores del desarrollo

4.1. Obtención de los datos iniciales

Gracias a la digitalización de recursos en las administraciones públicas hay una gran cantidad de datasets descargables con información relevante a el estudio que queremos realizar. Varios candidatos iniciales son:

- Instituto Geográfico Nacional: Contiene datasets muy específicos, con gran cantidad de delimitaciones geográficas, pero no se ajusta al 100% a los datos que queremos [20]
- Instituto Nacional de Estadística: La página web del INE cuenta con un centro de descargas de datos, pero carece de datos georreferenciados útiles para nuestro estudio.
- Consorcio Regional de Transportes de Madrid: Publica datos relativos al transporte público de la Comunidad de Madrid, como paradas de Metro, bus, cercanías o líneas de autobús.
- NOMECALLES: Portal web con visor de gran cantidad de datos georreferenciados de la Comunidad de Madrid.

Evaluando las opciones presentadas concluimos que el sitio web NOMECALLES, del Instituto de Estadística de la Comunidad de Madrid, contiene la gran mayoría de datasets que requiere nuestro estudio.

NOMECALLES nos permite visualizar y descargar distintos datasets georreferenciados de la Comunidad de Madrid, estos se dividen en Puntos y Delimitaciones, el primero ofrece datasets de puntos exactos y el segundo ofrece datasets de distintas divisiones territoriales y delimitaciones de todo tipo.

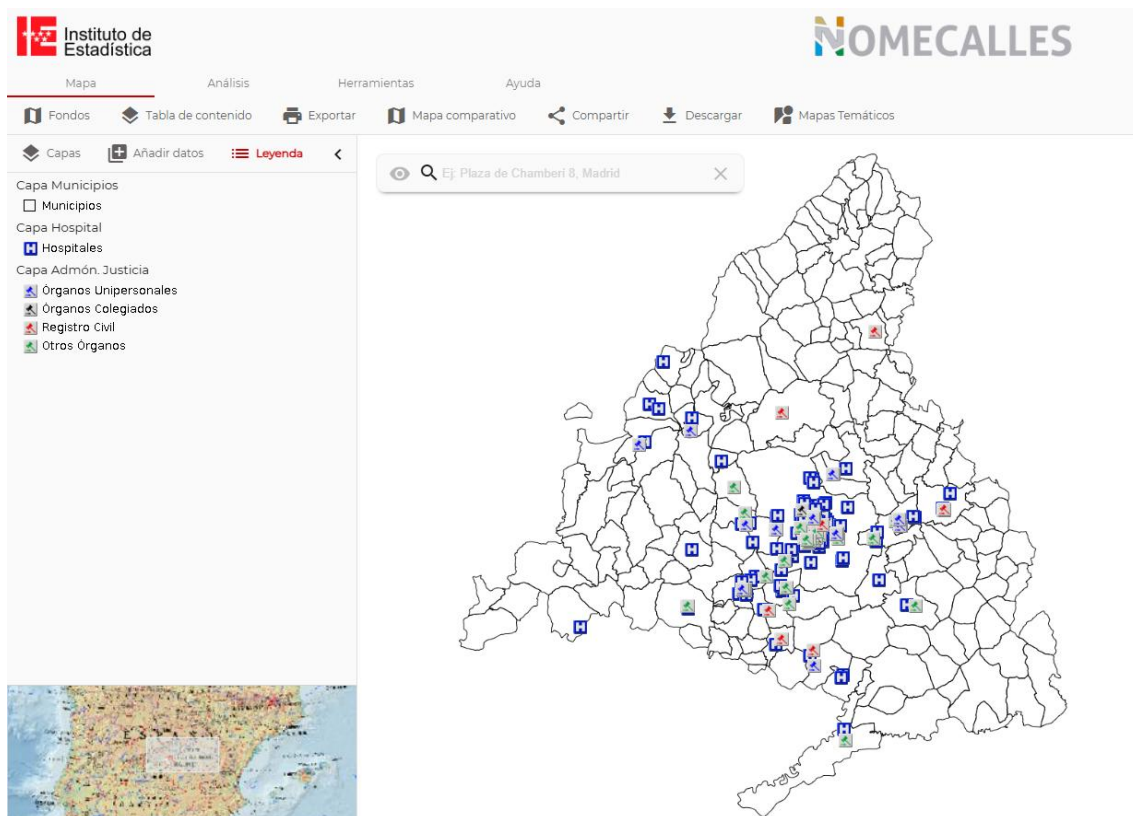


Ilustración 5 Captura de pantalla de la página web NOME CALLES

De todos los conjuntos de datos que ofrece la página extraemos los siguientes:

- Delimitación de municipios: Delimitaciones → Municipios
- Núcleos urbanos: Delimitaciones → Nomenclátor → Núcleos
- Dirección Asistencial de Salud (DAS): Delimitaciones → Zonificación de Salud de Área Única → Dirección Asistencial de Salud
- Juzgados: Puntos → Admón. Pública y Seguridad Ciudadana → Instituciones Públicas → Admón. Justicia
- Centros de Salud Mental: Puntos → Salud y Servicios Sociales → Centros Sanitarios → Centros de Salud Mental
- Hospitales: Puntos → Salud y Servicios Sociales → Centros Sanitarios → Hospital
- Parques de bomberos: Puntos → Admón. Pública y Seguridad Ciudadana → Protección y Seguridad Ciudadana → Bomberos

Algunos datasets georreferenciados fueron obtenidos de otras fuentes al no aparecer en el nomenclátor de NOME CALLES:

- Paradas de autobús EMT/Urbano/Interurbano: Obtenidas de la página del Consorcio de Transportes de Madrid [9]. Este dataset es necesario para obtener distancias utilizando transporte público
- Red de carreteras de Madrid: Versión modificada de dataset obtenido en el Centro de Descargas del Instituto Geográfico Nacional [20]. Este dataset es necesario para poder aproximar centroides a la carretera más cercana

Datos adicionales en formato tabla/CSV/XLSX

- Población por unidad poblacional: Obtenidas en el buscador del Insituto Nacional de Estadística [21]
- Listado de partidos judiciales de la Comunidad de Madrid obtenidas en la página web del Ministerio de Justicia de España

4.2. Preprocesado de datos

- Comprobación del sistema de referencia de coordenadas (CRS): Dos capas con CRS distintos pueden dar problemas si se realizan operaciones que afecten a ambas. Como primer paso se debe comprobar que todas las capas tienen el mismo CRS y en caso de que varíen, utilizar la herramienta “Reproyectar” para pasar todas las capas erróneas al CRS EPSG:25830. Este CRS abarca la mayoría de Europa, por lo que es perfecto para representar datos de la Comunidad de Madrid.
- Eliminación de geometrías inválidas: Algunos datasets contienen geometrías que dan problemas, como delimitaciones que no se cierran o líneas que no conectan. Con la herramienta “Corregir geometrías” se arregla fácilmente cualquier dataset siempre que la geometría no falle de forma excesiva.

Afortunadamente muy pocos datasets presentaban problemas. El dataset de municipios contenía partes de geometría inválida pero la herramienta permitió corregirlo fácilmente y el dataset de carreteras contenía un CRS distinto y pudimos reproyectarlo fácilmente.

Después de estos cambios, no consideramos realizar ningún cambio adicional en el formato de los datos, el formato de algunas cadenas de texto podían dar problemas al utilizar editores de texto pero dichos atributos no iban a ser utilizados en ninguna operación importante.

4.3. Procesado de datos

Una vez tenemos todos los datasets necesarios cargados en QGIS, podemos comenzar el proceso de transformación de los datos obtenidos.

Los datasets con los que comenzamos contienen los siguientes atributos

Dataset de núcleos poblacionales

Atributo	Descripción
CMUN	Código del municipio al que pertenecen (3 dígitos)
CDENTIDAD	Código de entidad (2 dígitos)
CDNUCLEO	Código de núcleo (2 dígitos)
CDTNUCLEO	Unión de los 3 atributos anteriores (7 dígitos)
DESCR	Nombre del núcleo poblacional en distintos estilos
BUSCA	
ETIQUETA	

Los atributos más importantes desde nuestro punto de vista es CDTNUCLEO. Es un identificador de cada núcleo poblacional, no se repite en ningún caso y nos permite identificar cada instancia de datos.

Explicación rápida del significado del atributo CDTNUCLEO: En la Ilustración 6 el municipio de Fresno de Torote está identificado por CMUN = 057. Dentro del municipio hay 3 entidades que se pueden diferenciar por el color en la parte derecha de la imagen Ilustración 6, identificadas por las 5 primeras cifras: 05701, 05702 y 05703. Dentro de estas entidades es posible que existan más de un núcleo poblacional, por lo que serían necesarias las últimas 2 cifras que, si son valores como 01, 02 o 03 indica que son núcleos poblacionales y si terminan en 99 indica que se trata de un diseminado.

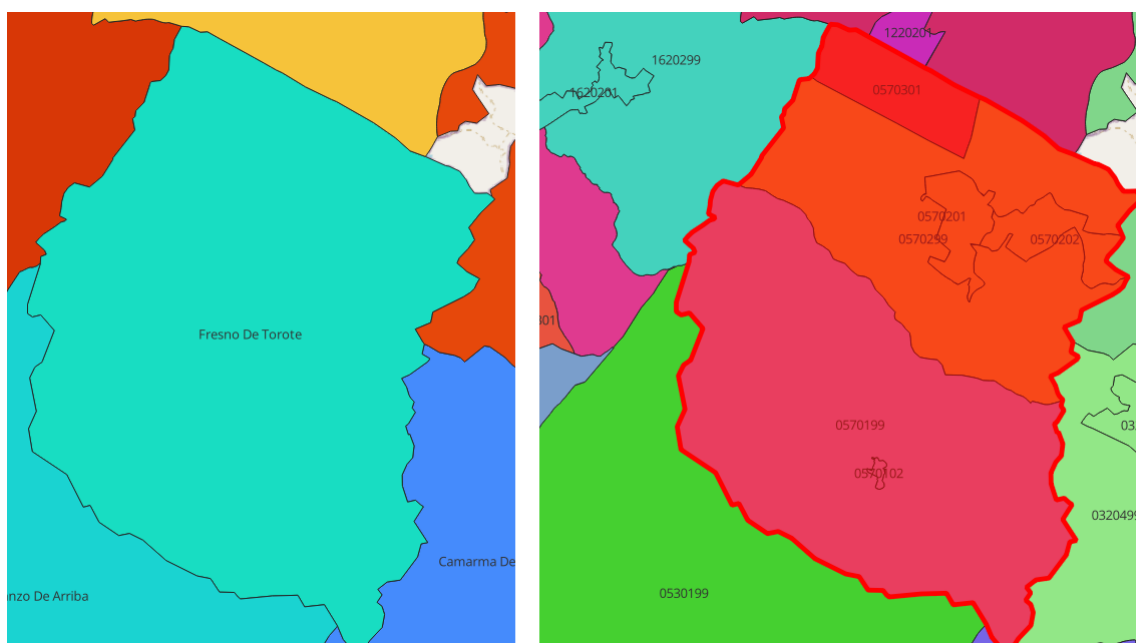


Ilustración 6 Ejemplo de núcleos poblacionales dentro del mismo municipio

Dataset de municipios

Atributo	Descripción
CMUN	Código del municipio
CODIGO	CMUN pero con una cifra adicional (redundante, no se utiliza en el estudio)
ETIQUETA	Nombre del municipio en distintos estilos
DESCR	

Datasets de servicios públicos

Los datasets de los distintos servicios públicos presentan una mayor cantidad de atributos, muchos de ellos no presentan información relevante para el estudio pero decidimos ignorarlos en vez de eliminarlos. A continuación presentamos los atributos más importantes de cada uno de estos datasets:

Hospitales

Atributo	Descripción
<u>CODIGO</u>	CHXXXX, CH seguido de 4 cifras que identifican al hospital
BUSCA/DESCR/ETIQUETA	Nombre del hospital en diferentes estilos
CMUN	Municipio al que pertenece

Centros de Salud Mental

Atributo	Descripción
<u>COD_SUCA</u>	Identificador de 8 cifras del Centro de Salud Mental
BUSCA/DESCR/ETIQUETA	Nombre del CSM en diferentes estilos

Juzgados

A parte de un atributo en específico, no encontramos ningún atributo por el que se pudiese indexar y que presentase información relevante. El único atributo por el que el dataset es indexable es BUSCA, que contiene el nombre completo del centro y la dirección del edificio.

Atributo	Descripción
<u>BUSCA</u>	Nombre completo del juzgado y dirección

Parques de bomberos

Atributo	Descripción
<u>COD_SUCA</u>	Identificador de 8 cifras del parque de bomberos
BUSCA/DESCR/ETIQUETA	Nombre del parque de bomberos en diferentes estilos

4.3.1. Filtrado de diseminados y núcleos con menos de 10 habitantes

El dataset de núcleos urbanos necesita varios ajustes y es que, aunque la definición de núcleo poblacional excluya a los diseminados, el dataset incluye ambos, probablemente por motivos estéticos y para cubrir todo el territorio de la comunidad. El propio dataset contiene información para que fácilmente se pueda saber si cualquier delimitación del dataset entra en cada una de estas dos categorías, por lo tanto, creamos una nueva capa que excluya todos los diseminados del conjunto.

Existen varias formas de realizar el filtrado, cualquier núcleo cuyo atributo CDTNUCLEO termine en “99” es un diseminado así que filtrando esos elementos fuera obtenemos una capa sin diseminados. Otra forma de filtrado, que salta bastante a la vista, sería filtrar todos los núcleos poblacionales con “(Diseminado)” en sus atributos BUSCA, DESCR o ETIQUETA.

El filtrado se realiza mediante expresiones en formato query de SQL [22] , así que una expresión válida para nuestro caso es

```
("DESCR" NOT LIKE '%Diseminado%') AND  
("BUSCA" NOT LIKE '%Diseminado%') AND  
("ETIQUETA" NOT LIKE '%Diseminado%')
```

Con esta consulta te aseguras de que ningún resultado contenga el substring “Diseminado” en ninguno de los atributos en los que lo puede tener. La ejecución de esta consulta filtra nuestro dataset de 1208 filas nos devuelve un dataset con 571 elementos y reduce enormemente el terreno abarcado.

A continuación, consultamos los datos del INE sobre población por unidad poblacional [21], y obtenemos un listado en formato CSV de todos los núcleos poblacionales d Madrid y su censo. Este CSV contiene los atributos CMUN, CIDENTIDAD y CDNUCLEO, por lo que es fácil crear una nueva columna con Excel que una estas 3 columnas, creando el equivalente a CDTNUCLEO.

QGIS, como mencionamos previamente también permite añadir y visualizar archivos CSV en el proyecto, una vez añadido se puede unir con nuestra capa de núcleos poblacionales mediante un índice que en nuestro caso es CDTNUCLEO, añadiéndole a esta un nuevo atributo con el número de población, a la que más tarde le llamaremos POB.

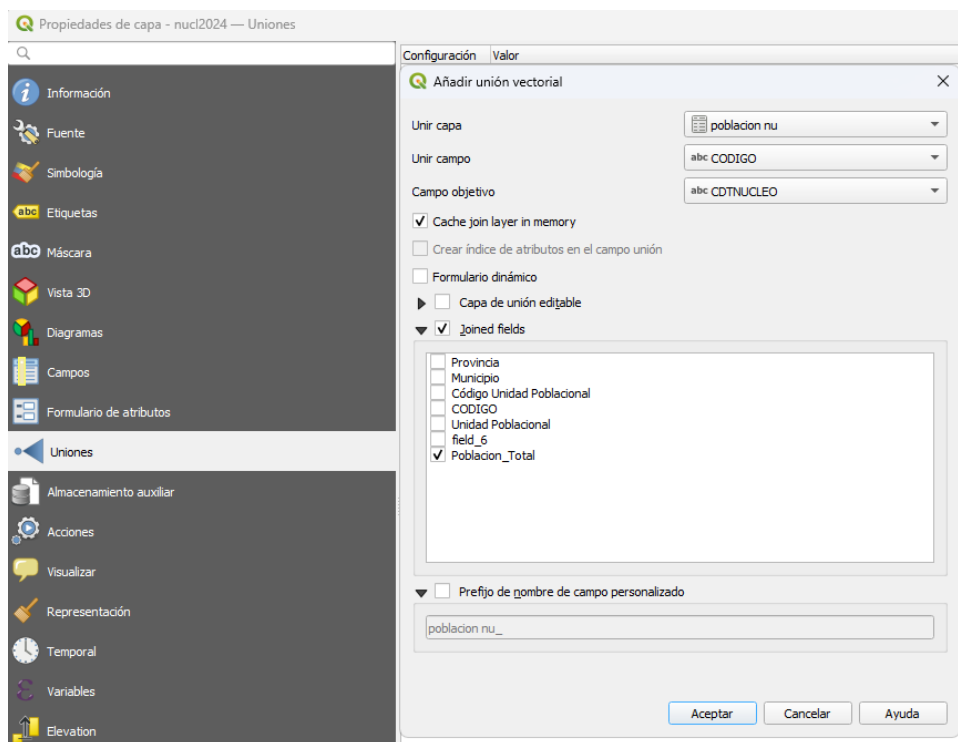


Ilustración 7 Ejemplo de uso de la función de Unión en QGIS

A esta nueva capa se le puede aplicar un nuevo filtrado, quedándonos con sólo los núcleos poblacionales con más de 9 habitantes. Tomamos esta decisión porque la gran

mayoría de núcleos poblacionales con menos de 10 habitantes carecen de conexión por carreteras convencionales y pueden dar problemas en pasos posteriores. Este filtrado se hace con la expresión

“POB”>=10

Sin embargo podemos ver en los datos 6 núcleos poblacionales cuyo atributo POB es NULL. Esto se debe a que el INE no tiene datos sobre su población pero el Instituto de Estadística de Madrid los considera como núcleos. No existen registros de población sobre dichos núcleos, por lo que decidimos prescindir de ellos, estos son:

- Cañada del Molino
- Centro Penitenciario Madrid III
- Cercas de la Encina
- Valdemera
- Umbría de Tórtolas
- Quintana

Esta última operación de filtrado deja el dataset con un total de 468 elementos

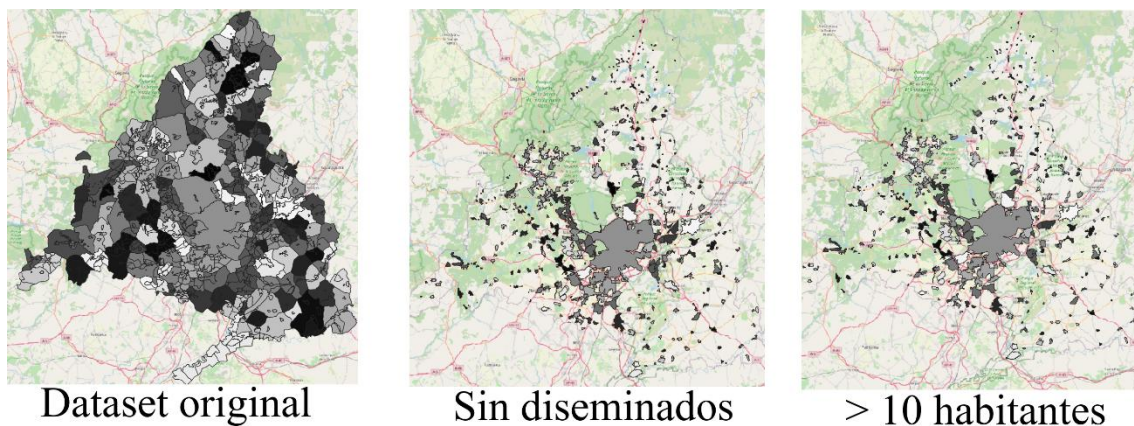


Ilustración 8 Proceso de filtrado de núcleos poblacionales

4.3.2. Obtención de los centroides de los núcleos poblacionales

Como se puede observar, este dataset contiene datos georreferenciados de cada núcleo poblacional en forma de delimitaciones geográficas. Esta representación es importante pero no nos sirve para las operaciones que debemos llevar a cabo.

En pasos posteriores es necesario establecer un punto de origen y un punto de destino para calcular distancias en la Routes API de Google, para esto debemos designar a cada núcleo poblacional un punto que sirva como referencia. Este punto resultante obviamente heredará todos los atributos del dataset original para los pasos posteriores.

Originalmente se planteó el uso de la herramienta Centroides, pero rápidamente se puede llegar a la conclusión de que muchos núcleos poblacionales tienen formas no convexas, haciendo que el centroide caiga fuera del núcleo.

Ilustración 9 Ejemplo de centroides en núcleos poblacionales no convexos

Para evitar futuras complicaciones elegimos utilizar la herramienta Punto en superficie, la cual devuelve una dataset de puntos que, aunque no sean centroides de los núcleos poblacionales, está garantizado que se encuentra dentro de la delimitación de este.

En la Ilustración 9 se puede ver la diferencia entre los puntos resultantes de una herramienta y otra en núcleos poblacionales con formas no convexas y geometrías más complejas, el punto rojo es el resultante de la herramienta Centroides y queda completamente fuera de la delimitación del núcleo, en cambio el punto verde se obtiene con la herramienta Punto en superficie y es notablemente más representativo del núcleo



En pasos posteriores se tuvo que hacer backtracking a este punto para añadir varios cambios al dataset que en su momento no se contemplaron pero se necesitan para pasos posteriores:

Ajuste de centroides a la carretera más cercana

En el punto 4.1. Obtención de los datos iniciales mencionamos un dataset que contiene la red de carreteras de la Comunidad de Madrid publicado por el Instituto Geográfico Nacional. Este dataset lo debemos procesar ligeramente para poder ajustar los centroides del paso anterior a la carretera más cercana a ellos. Esto lo hacemos para corregir centroides que se hayan generado en zonas despobladas y sin red de carreteras cercana, asegurándonos de que en pasos posteriores no genere ningún tipo de fallo:

El primer paso para procesar esta red de carreteras es quedarnos solo con las carreteras dentro de los núcleos poblacionales, esto se consigue utilizando la herramienta Cortar que elimina cualquier geometría fuera de la delimitación de los núcleos poblacionales. Como se puede ver en la Ilustración 10, queda una capa más limpia y sin geometrías que no vamos a utilizar.

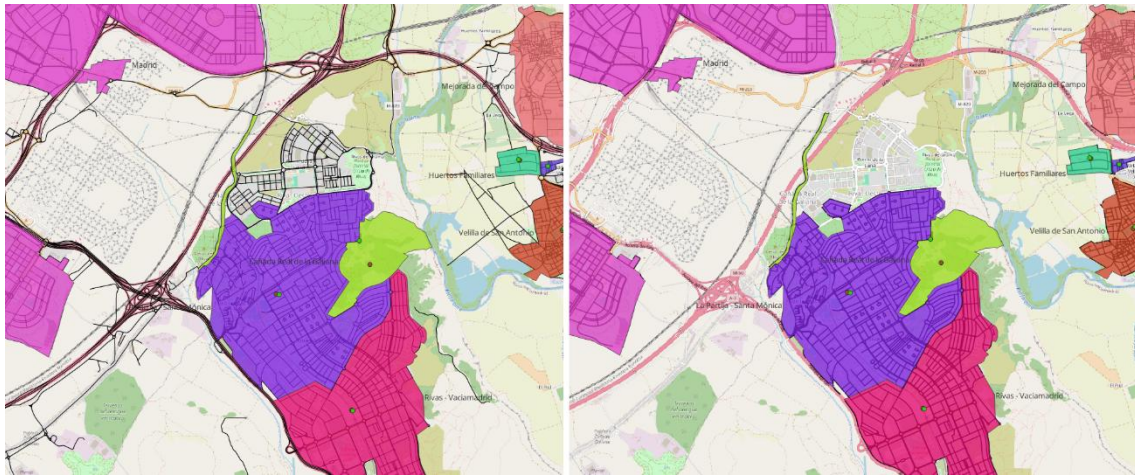


Ilustración 10 Carreteras antes y después del uso de la herramienta Cortar

El siguiente paso es mover el centroide de cada núcleo poblacional a la carretera más cercana dentro de los límites del núcleo, esta operación a simple vista parece sencilla, ya que QGIS tiene una herramienta llamada “Ajustar geometrías a capa” que permite mover puntos de una capa al elemento más cercano de otra capa.

Sin embargo, la operación que queremos realizar viene con una condición extra: La carretera a la que se ajusta no debe estar en otro núcleo poblacional. Esta condición no parece muy relevante, pero exige una operación que no permite ninguna herramienta de QGIS, por lo que utilizamos un plugin externo que nos lo permita.

El primer paso para este proceso es el uso de la herramienta “Unir atributos por localización”, añadiendo a cada carretera el atributo CDTNUCLEO del núcleo poblacional en el que se encuentra, esto permite fácilmente identificar si una carretera está en el mismo núcleo.

El segundo paso es el uso del plugin externo ProcessX [23], el cual cuenta con una herramienta llamada “Snap Vertices to Nearest Points by Condition” semejante a “Ajustar geometrías a capa”, pero que permite añadir la condición de que solo se ajuste si comparten un atributo.

En la Ilustración 11 se puede ver un ejemplo del funcionamiento de esta herramienta y por qué es necesaria: El punto naranja es el centroide obtenido para el núcleo poblacional rojo, la herramienta “Ajustar geometrías a capa” ajustaría el punto a la carretera más cercana, señalada por la línea roja, la cual pertenece a otro núcleo poblacional, mientras que con la herramienta “Snap Vertices to Nearest Points by Condition” el punto verde resultante se encuentra realmente en la carretera más cercana dentro del mismo núcleo poblacional, exactamente lo que necesitamos.

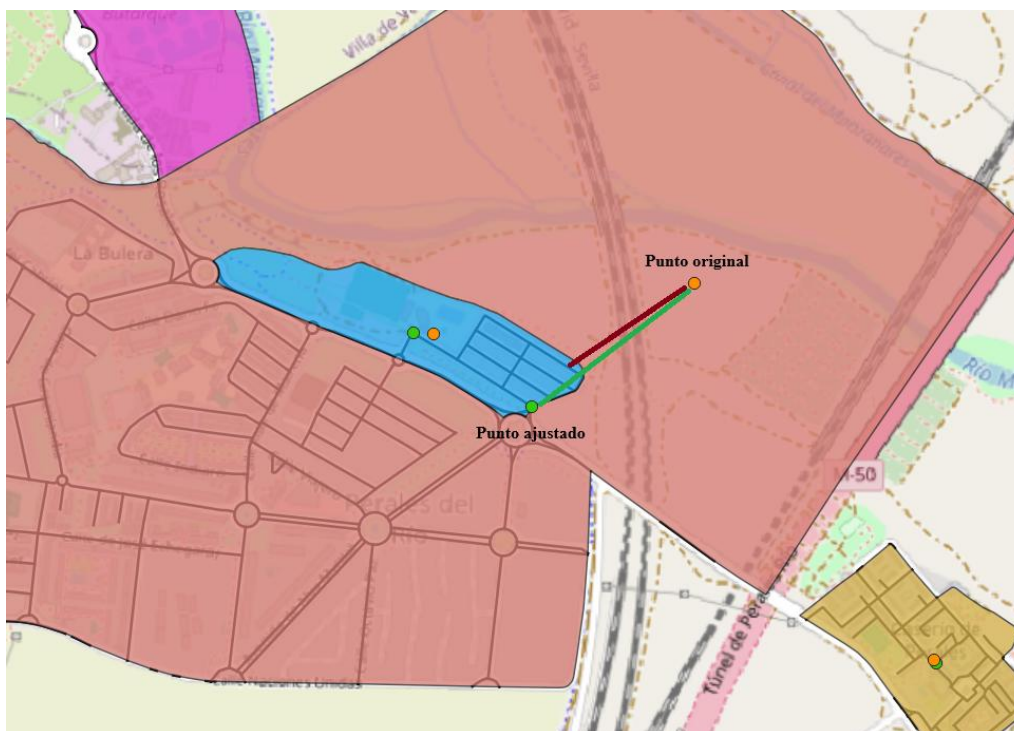


Ilustración 11 Ajuste de centroide a carretera

Desplazamiento manual de centroides puntuales

Un total de núcleos poblacionales se encuentran prácticamente incomunicados en lo que respecta a transporte público, implicando que es posible una ruta que utiliza transporte público, pero conlleva un trayecto a pie demasiado largo hasta la parada de metro o bus. Routes API de Google a la hora de calcular un trayecto, si uno de los puntos no está cerca de una carretera calcula el tiempo que tardaría en llegar andando a la parada de transporte público más cercana, pero esta “tolerancia” a puntos en posiciones atípicas parece ser limitada.

La cantidad de núcleos afectados es lo suficientemente pequeña como para poder hacer la modificación de la posición de los núcleos a mano. Después de este proceso, calculamos mediante la función `get_penalization()` de Python las distancias de los núcleos desplazados para poder utilizar esta información más tarde y poder tener en cuenta que estos núcleos realmente tienen una distancia y duración de viaje mayor a la que refleja el resultado.

```
def get_penalization(old, new):
    old_layer = project.mapLayersByName(old)[0]
    old_features = list(old_layer.getFeatures())
    new_layer = project.mapLayersByName(new)[0]
    new_features = list(new_layer.getFeatures())

    penalizations = {}

    for old_feature in old_features:
        for new_feature in new_features:
            if old_feature["CDTNUCLEO"]==new_feature["CDTNUCLEO"]:
                dist = qgis.QgsDistanceArea().measureLine(
                    old_feature.geometry().centroid().asPoint(),
                    new_feature.geometry().centroid().asPoint()
                )
                if dist>0:
                    penalizations[old_feature["CDTNUCLEO"]] = dist
                    continue

    return penalizations
```

Código 1 Función get_penalizations()

Los núcleos poblacionales a los que se les aplica este cambio y sus distancias de desplazamiento son los siguientes:

Núcleo	Desplazamiento (m)	Núcleo	Desplazamiento (m)
Las Herreras:	2630	Los Rancajales	3459
La Pizarrera	2186	Costa de Madrid	3889
Las Cuestas	2441	Javacruz	2893
Santa Cruz del Valle de los Caídos	4907	San Ramón	3090
Valle de los Caídos	3701	Las Hoyas	5107
Los Tomillares	4224	Santa Ana	3423
La Berzosilla	3425	El Morro	3100
Carracalderas	2127	Valbosque	4224
Los Chortales	4503	Valdeperales	2161
Los Endrinales	2804	Alarilla	4241

4.3.3. Filtrado de hospitales

El dataset de hospitales proporcionado por NOMEALLES contiene centros hospitalarios con enorme disparidad de tamaño y complejidad. Por lo tanto, no podemos considerar de la misma manera un núcleo de población cuyo hospital más cercano es un hospital de la complejidad más baja y un núcleo de población a la misma distancia del hospital Gregorio Marañón.

Por esto mismo, dividimos este dataset en otros dos nuevos conjuntos de datos, esta división la hacemos acorde a la complejidad de los hospitales y, como aparece reflejada en el dataset de NOMEALLES, se hace de forma manual guiándonos por la información sobre centros hospitalarios de la Red del Servicio Madrileño de Salud de la página web de la Comunidad de Madrid [24].

Los dos datasets resultantes son:

- Hospitales de grupo 3: Hospitales de gran complejidad.
- Hospitales de grupo 2: Hospitales de complejidad intermedia

En la siguiente tabla se puede ver la lista de hospitales que pasaron el filtro y que, por lo tanto, serán utilizados en los siguientes pasos:

Hospitales de grupo 3

Nombre	Dirección Asistencial de Salud
Hospital Universitario 12 de Octubre	Centro
Hospital Universitario de la Princesa	Centro
Hospital Universitario Fundación Jiménez Díaz	Noroeste
Hospital Clínico San Carlos	Noroeste
Hospital Universitario Puerta de Hierro Majadahonda	Noroeste
Hospital Universitario La Paz	Norte
Hospital Universitario Ramón y Cajal	Norte
Hospital General Universitario Gregorio Marañón	Sureste

Hospitales de grupo 2

Nombre	Dirección Asistencial de Salud
Hospital Centro Sanitario de Vida y Esperanza	Centro
Hospital Universitario Príncipe de Asturias	Este
Hospital Universitario de Torrejón	Este
Hospital General de Villalba	Noroeste
Hospital Universitario Infanta Sofía	Norte
Hospital Universitario de Móstoles	Oeste
Hospital Universitario Fundación Alcorcón	Oeste
Hospital Universitario Fuenlabrada	Oeste
Hospital Rey Juan Carlos	Oeste
Hospital Universitario Severo Ochoa	Sur
Hospital Universitario de Getafe	Sur
Hospital Universitario Infanta Leonor	Sureste

4.3.4. Adición y herencia de atributos

El dataset en este momento contiene todos los destinos (hospitales, juzgados, parques de bomberos y centros de salud mental), los municipios y los núcleos poblacionales con sus respectivos centroides ajustados. Pero aún se necesitan varios atributos que faciliten los pasos posteriores, además de brindar información adicional que puede ser relevante.

Dirección Asistencial de Salud (DAS)

El primer atributo adicional es la Dirección Asistencial de Salud. En Madrid existen 7 DAS y estas delimitaciones determinan cuál es el hospital de referencia de cada núcleo poblacional, esto evita que se calcule la distancia de un núcleo poblacional a un hospital de una DAS distinta.

Cada DAS tiene, por lo menos, un hospital de grupo 2, por lo que el hecho de la existencia de un hospital de grupo 3 dentro de la misma DAS se considerará como un punto a favor al calcular la disponibilidad de un núcleo poblacional.

Cada DAS se representará en los datasets por el atributo **COD**, con un número del 1 al 7. La herramienta “Unir atributos por localización” permite que un punto/delimitación de una capa herede el atributo de la delimitación de otra capa siempre que lo interseque o lo contenga. Esta herramienta se utiliza en la capa de centroides de los núcleos urbanos, la capa de hospitales de grupo 3 y de hospitales de grupo 2. En la Ilustración 12 se puede ver la delimitación de cada DAS y cómo los núcleos y hospitales dentro de cada uno conserva el atributo COD de dicho DAS.

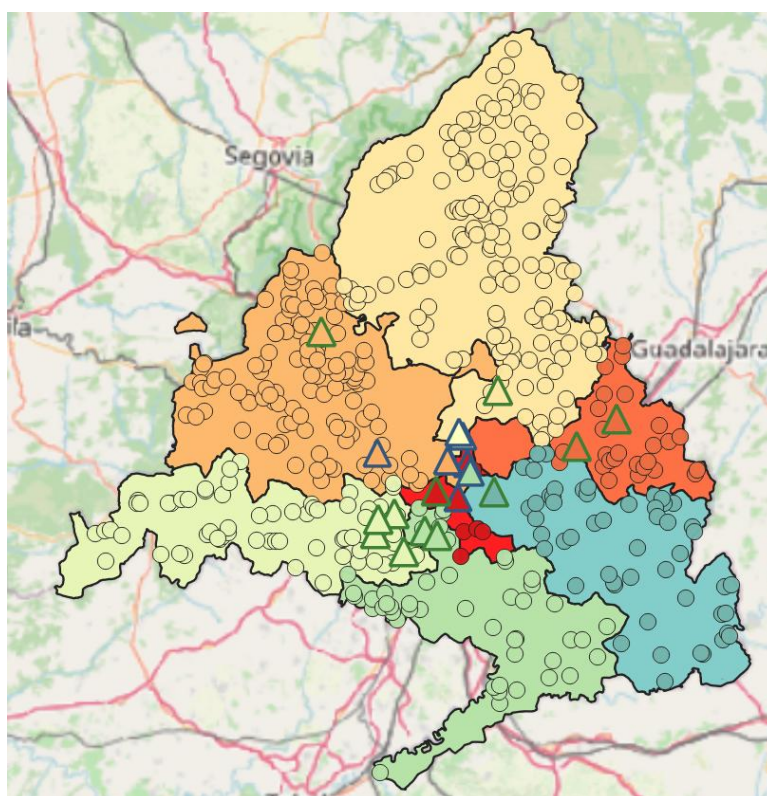


Ilustración 12 Núcleos y hospitales clasificados por su DAS

Partido judicial

El segundo atributo que añadir es el **partido judicial**. Según el Artículo 32 de la Ley Orgánica del Poder judicial [25], el partido judicial es una división de territorios uno o más municipios limítrofes, pertenecientes a una misma provincia. Esta división la utilizaremos para que un núcleo urbano no detecte como juzgado más cercano a uno de un partido judicial distinto.

Por lo tanto, se añade un atributo que determine a cuál de los 21 partidos judiciales pertenece cada núcleo de población y cada juzgado, acorde a los listados proporcionados por el Ministerio de Justicia [26] [27]. En la Ilustración 13 se representa una clasificación de núcleos poblacionales (círculos) y juzgados (cuadrados) por el partido judicial al que pertenecen.

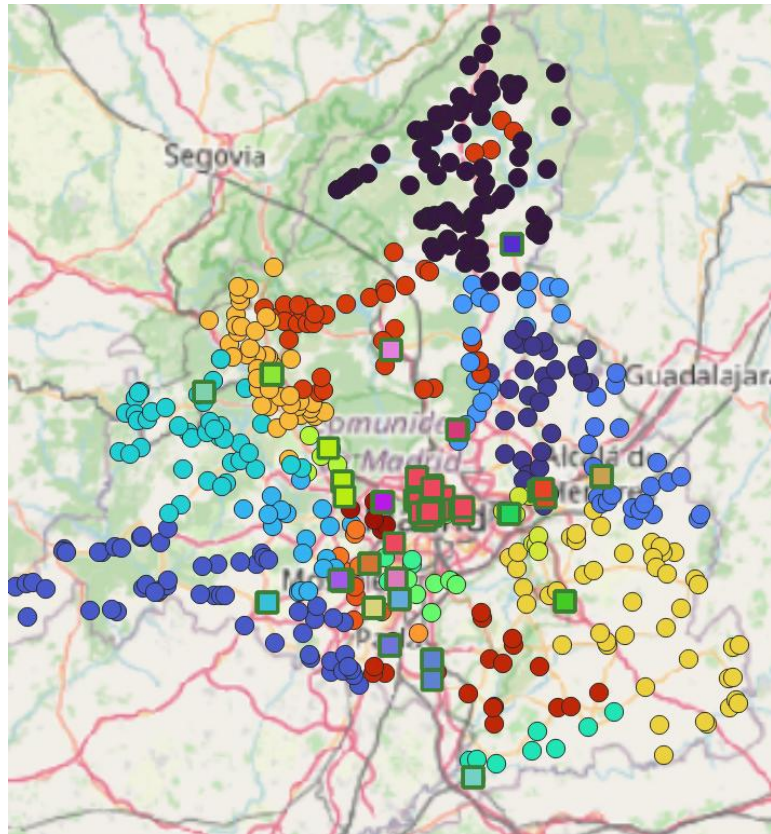


Ilustración 13 División por partido judicial

Latitud y Longitud

Atributos **lat** y **lng**. Contienen la latitud y longitud de cada punto. Estos atributos se pueden obtener en este punto del trabajo utilizando la herramienta Añadir campos X/Y a capa, pero se puede omitir completamente ya que utilizando la librería de QGIS para Python se puede obtener de forma rápida, pero es información adicional que puede ser útil en cualquier dataset con datos georreferenciados

4.4. Selección de vecinos más cercanos a través de la librería de QGIS para Python

QGIS ofrece su propia librería de Python, `qgis.core`, para realizar operaciones sobre capas de archivos `.qgz`, que contienen el proyecto QGIS completo.

El desarrollo de la mayor parte de funciones que interactúen con el proyecto QGIS o creen archivos nuevos se realiza en el archivo Python `file_creator.py`.

Antes de cada ejecución se debe realizar un setup que permita la lectura correcta del proyecto QGIS cargando las capas del proyecto en memoria en una variable llamada `project`.

```
import qgis.core as qgis
qgis.QgsApplication.setPrefixPath("<Path_a_QGIS>", True)
qgs = qgis.QgsApplication([], False)
qgs.initQgis()
project = qgis.QgsProject.instance()
```

Código 2 Setup usando la librería `qgis.core` en Python

4.4.1. Métodos `closest_destinations_cords()` y `closest_destinations_features()`

Esta función toma solo dos parámetros:

- **origin**: String con el nombre de la capa de origen, es decir, de los centroides de núcleos poblacionales.
- **destinations**: Lista con los nombres de las capas de hospitales, parques de bomberos, juzgados y centros de salud mental.

Antes de la ejecución de la función se definen varias variables.

`project` es, de forma resumida el proyecto QGIS donde se encuentran todas las capas.

El funcionamiento del algoritmo es, traducida a lenguaje natural:

1. Crea una lista con todos los puntos de origen y, para cada tipo de servicio público, una lista de todos los puntos de destino.
2. Itera de forma que, para cada núcleo compruebe la distancia a cada punto dependiendo de las siguientes condiciones dependiendo del tipo de servicio público:
 - **Hospitales**: Para cada núcleo poablacional las coordenadas de todos los hospitales de grupo 2 y 3 por separado que pertenezcan al mismo DAS.
 - **Juzgados**: Filtra todos los juzgados que no estén en el mismo partido judicial y luego calcula la distancia euclídea a cada uno de ellos, guardando las coordenadas sólo del juzgado más cercano.
 - **Parques de bombero**: No hace filtrado, por lo que calcula todas la distancia desde el núcleo a todos los parques de bomberos y guarda las coordenadas del más cercano.

- Centros de salud mental: Filtra para cada núcleo los CSM del mismo DAS y calcula la distancia para quedarse con las coordenadas del más cercano
3. Devuelve un diccionario de Python que, para cada núcleo poblacional, contiene las coordenadas del propio núcleo y las coordenadas de los destinos.

```
def closest_destinations_cords(origin: str, destinations: list[str]) -> dict[str,dict[str,list[str]]]:
    """
    Lee el proyecto QGIS y devuelve un diccionario
    Para cada origen guarda sus coordenadas y las coordenadas de los destinos más cercanos
    """
    closest = {}
    origin_layer = project.mapLayersByName(origin)[0]
    origin_features = list(origin_layer.getFeatures())
    destination_layers = [project.mapLayersByName(dest)[0] for dest in destinations]
    destination_features = [list(layer.getFeatures()) for layer in destination_layers]
    crs_src = origin_layer.crs()
    crs_dest = qgis.QgsCoordinateReferenceSystem("EPSG:4326")
    transform = qgis.QgsCoordinateTransform(crs_src, crs_dest, qgis.QgsProject.instance())
    for origin_feat in origin_features:
        origin_id = origin_feat[identifiers[origin_layer.name()]]
        origin_point = origin_feat.geometry().asPoint()
        origin_ll = transform.transform(origin_point)
        origin_cords = [origin_ll.y(),origin_ll.x()]
        closest[origin_id]={"cords":origin_cords, "destinations":[]}
        for i, dest_layer_feats in enumerate(destination_features):
            dest_name = destinations[i]
            filtered_dest_feats = []
            # Filtra destinos
            if dest_name in ["Hospitales grupo 2", "Hospitales grupo 3", "Salud mental"]:
                if 'COD' in origin_feat.fields().names():
                    origin_cod = origin_feat['COD']
                    filtered_dest_feats = [f for f in dest_layer_feats if f['COD'] ==
origin_cod]
                elif dest_name == "Juzgados":
                    if 'PART_JUD' in origin_feat.fields().names():
                        origin_part_jud = origin_feat['PART_JUD']
                        filtered_dest_feats = [f for f in dest_layer_feats if f['PART_JUD'] ==
origin_part_jud]
                else:
                    filtered_dest_feats = dest_layer_feats

            if dest_name in ["Hospitales grupo 2", "Hospitales grupo 3"]:
                added = [[transform.transform(f.geometry().asPoint()).y(),
                    transform.transform(f.geometry().asPoint()).x()]
                    for f in filtered_dest_feats]
                closest[origin_id]["destinations"].append(added)
                continue
            # Busca el destino más cercano en caso de que no sean hospitales
            min_dist = float('inf')
            min_feat = None
            for dest_feat in filtered_dest_feats:
                dist = qgis.QgsDistanceArea().measureLine(
                    origin_feat.geometry().centroid().asPoint(),
                    dest_feat.geometry().centroid().asPoint()
                )
                if dist < min_dist:
                    min_dist = dist
                    min_feat = dest_feat
            # Guardamos sus identificadores
            pt = transform.transform(min_feat.geometry().asPoint())
            closest[origin_id]["destinations"].append([pt.y(), pt.x()])
    return closest
```

Código 3 Método closest_destinations_features()

El método closest_destinations_features() hace exactamente lo mismo pero en las últimas tres líneas, el diccionario guarda los nombres de los destinos y orígenes. Es útil

principalmente a la hora de comprobar que funciona correctamente

```
None)
    # Guardamos sus identificadores
    closest_feats.append(min_feat[identifiers[dest_name]] if min_feat else
    closest[origin_id] = closest_feats
    return closest
```

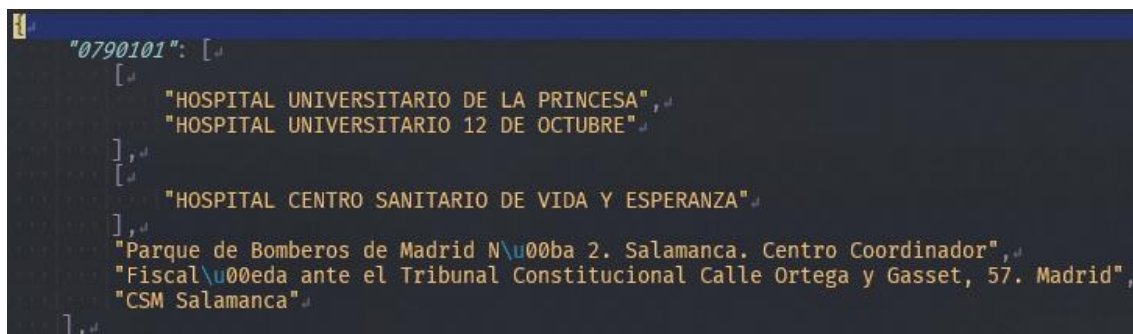
Código 4 Método closest_destinations_cords()

El resultado de la función `closest_destinations_cords()` devuelve un diccionario que guardamos como archivo JSON con el formato que se puede ver en la Ilustración 14. Este JSON es clave para los pasos posteriores y es utilizado para realizar las llamadas a Routes API en siguientes pasos, ya que contiene de forma clara todas las coordenadas a utilizar.

```
{
  "0790101": {
    "cords": [
      40.42323536185605,
      -3.6629893725735445
    ],
    "destinations": [
      [
        40.43448605632367,
        -3.6758233245532175
      ],
      [
        40.37645105789588,
        -3.6968357352994903
      ],
      [
        40.38818516459446,
        -3.745274506608776
      ],
      [
        40.42855384810598,
        -3.667252259059084
      ],
      [
        40.43006735942887,
        -3.675012784351704
      ],
      [
        40.42153900790907,
        -3.6713676135003244
      ]
    ]
  }
}
```

Ilustración 14 Fragmento del output de closest_dest_cords()

La función `closest_destinations_features()` devuelve un diccionario convertido en JSON con el formato mostrado en la Ilustración 15. Este diccionario no es utilizado en pasos posteriores del estudio pero aporta una vista fácil de interpretar del output de la función, permitiendo una mayor facilidad en el debugging.



```
"0790101": [
  [
    "HOSPITAL UNIVERSITARIO DE LA PRINCESA",
    "HOSPITAL UNIVERSITARIO 12 DE OCTUBRE"
  ],
  [
    "HOSPITAL CENTRO SANITARIO DE VIDA Y ESPERANZA"
  ],
  "Parque de Bomberos de Madrid N\u00ba 2. Salamanca. Centro Coordinador",
  "Fiscal\u00eda ante el Tribunal Constitucional Calle Ortega y Gasset, 57. Madrid",
  "CSM Salamanca"
]
```

Ilustración 15 Fragmento del output de `closest_dest_features()`

4.4. Consulta a la Routes API de Google

Una vez tenemos la lista completa con cada uno de los núcleos poblacionales relevantes para el estudio, planteamos un plan de consultas a la Routes API de Google Maps Platform [19], esta API devuelve, entre otras cosas, la distancia real entre dos puntos.

Para obtener un resultado más preciso, para cada par origen-destino, pedimos que devuelva la distancia y tiempo de recorrido con estos parámetros:

- En hora punta (8:30) en coche
- En hora punta en transporte público
- En hora con menos tráfico (11:00) en coche
- En hora con menos tráfico en transporte público

Con estos cuatro resultados nos aseguramos de que se tienen en cuenta distintas situaciones que también representan la disponibilidad de un servicio público.

Establecidas estos parámetros, dado que después del cálculo de vecinos más cercanos concluimos que existen 2867 pares origen-destino, el total de consultas a realizar asciende a 11468.

Métodos para la realización de consultas

Ya comentamos el uso del archivo `file_creator.py` para alojar todas las funciones que tuviesen que ver con creación de archivos o accesos al proyecto de QGIS en el que se encuentran todas las capas del estudio. La realización de consultas requiere el uso de distintas librerías y funciones completamente diferentes.

En el archivo `google_request.py` se encuentra cualquier método relacionado con consultas a la API y manejo de las respuestas de dichas consultas.

Teniendo en cuenta que se van a realizar 11468 consultas, ejecutar estas consultas secuencialmente daría como resultado ejecuciones extremadamente largas. Cada consulta implica un tiempo de espera que puede ser asumible en consultas atómicas

pero extremadamente tedioso con baterías de consultas tan grandes. Debido a esto se toma la decisión de desarrollar las funciones de consulta de forma asíncrona, permitiendo el envío de muchas consultas de forma paralela y reduciendo notablemente el tiempo de ejecución.

Los dos métodos principales del programa son `request_routes_v2_async()` y `fetch_route()`.

- `request_routes_v2_async()` toma un solo parámetro booleano “*preserve*”. *preserve* determina si ya se han realizado consultas previas a Routes API y se quieren realizar sólo las consultas de las cuales aún no se tengan los datos proporcionados por la API. La inclusión de este parámetro permite que en casos de fallos de conexión puntuales o respuestas erróneas no nos vamos obligados a realizar todas las consultas de nuevo y se pueden conservar las consultas previas. El resto del método hace llamadas asíncronas a la función `fetch_route()` y toma los resultados devueltos por dicha función, los almacena y los ordena en forma de diccionario para ser utilizado a posteriori.
- `fetch_route()`, en cambio, es el método que lleva a cabo todas las consultas, prepara la cabecera y cuerpo de cada consulta, notifica en caso de recibir una respuesta correcta o un mensaje de error y devuelve dicho resultado, así como el ID del origen, el ID del destino, el modo (en coche o transporte público) y la hora de la consulta, todo necesario para ensamblar los resultados en la función `request_routes_v2_async()`.


```

async def request_routes_v2_async(preserve=False):

    path = Path("output/closest_destinations_cords.json")
    with open(path, 'r') as file:
        json_file = json.load(file)

    if preserve:
        with open("output/routes_API_results_dump.json", 'r') as file:
            full_result = json.load(file)
    else:
        full_result = {}

    tasks = []

    for origin in json_file:
        if not preserve:
            full_result[origin] = []

        origin_cords = json_file[origin]["cords"]
        destinations = []
        for item in json_file[origin]["destinations"]:
            if item:
                if isinstance(item[0], list):
                    destinations.extend(item)
                else:
                    destinations.append(item)

            for dest_index, destination in enumerate(destinations):
                for mode in modes:
                    for hour_index, hour in enumerate(hours):
                        if preserve:
                            if not
full_result[origin][dest_index][f"{mode}_{hours_true[hour_index].strftime("%H:%M")}"] =
                            tasks.append(
                                fetch_route(origin_cords, destination, mode, hour, origin,
dest_index)
                            )
                        else:
                            tasks.append(
                                fetch_route(origin_cords, destination, mode, hour, origin,
dest_index)
                            )

        results = await asyncio.gather(*tasks)

        # Reconstruir el resultado en estructura original
        for origin_id, dest_index, mode, hour, data in results:
            hour_key = hours_true[hour_index].strftime('%H:%M')
            key = f"{mode}_{hour_key}"

            if len(full_result[origin_id]) <= dest_index:
                full_result[origin_id].extend([{}]) * (dest_index - len(full_result[origin_id]) +
1))

            full_result[origin_id][dest_index][key] = data

    return full_result

```

Código 5 Método request_routes_v2_async

```

async def fetch_route(origin_cords, destination, mode, hour, origin_id, dest_index):
    async with semaphore:
        body = {
            "origin": {"location": {"latLng": {"latitude": origin_cords[0], "longitude":
origin_cords[1]}},
            "destination": {"location": {"latLng": {"latitude": destination[0], "longitude":
destination[1]}},
            "travelMode": mode.upper(),
            "departureTime": hour.strftime("%Y-%m-%dT%H:%M:%SZ")
        }
        if mode == "drive":
            body["routingPreference"] = "TRAFFIC_AWARE"

        for attempt in range(3): # Reintenta hasta 3 veces
            try:
                async with httpx.AsyncClient(timeout=30) as client:
                    print(f"Enviando {origin_id}, Destino {dest_index} [{mode}
{hour.strftime('%H:%M')}]")
                    response = await client.post(URL, headers=HEADERS, json=body)
                    response.raise_for_status()
                    print(f"Recibido: {origin_id}, Destino {dest_index} [{mode}
{hour.strftime('%H:%M')}]")
                    return (origin_id, dest_index, mode, hour, response.json())
            except httpx.HTTPStatusError as e:
                print(f"HTTP {e.response.status_code} en {origin_id}-{dest_index}-{mode},
intento {attempt+1}")
                await asyncio.sleep(1)
            except Exception as e:
                print(f"Error en {origin_id}-{dest_index}-{mode}: {e}")
                await asyncio.sleep(2)
        return (origin_id, dest_index, mode, hour, {"error": "intentos agotados"})

```

Código 6 Método fetch_route()

El resultado obtenido es un diccionario que fácilmente se puede convertir en archivo JSON con la siguiente estructura:

```

{
    "0790101": [ ← CDTNUCLEO del origen
        {
            "drive_08:30": { ← Tipo de trayecto + hora
                "routes": [
                    {
                        "distanceMeters": 1929,
                        "duration": "464s"
                    }
                ]
            }, ← Aquí termina el resultado en coche a las 8:30
            ...
        }, ← Aquí terminan los resultados del primer destino
        ...
    ]
}

```

4.5 Depurado de los resultados

Una vez se tienen los resultados en formato .json hace falta aplicarle varias capas de modificaciones para obtener lo que consideraríamos el dataset completo.

En el punto 4.2 mencionamos que ciertos núcleos tienen que ser movidos a una parada de bus cercana para poder obtener resultados en las consultas a Routes API. Pues ahora es el momento en el que es necesario tener en cuenta qué núcleos fueron desplazados y cuánto desplazamiento se le aplicó.

El desplazamiento se aplicó en una copia de la capa original, así que, teniendo ambas capas podemos calcular la distancia entre el mismo punto en estas dos capas obteniendo una distancia euclídea. Esto lo obtenemos fácilmente con las funciones `get_penalization()` y `apply_penalization()`

El funcionamiento general de estas funciones es: `get_penalization()` guarda todos los núcleos desplazados y su desplazamiento en distancia euclídea, `apply_penalizaciones()` toma esas penalizaciones y las aplica a todos los núcleos afectados.

Esta penalización es una aproximación y simula el camino andando entre la parada de bus y el destino, dado que la distancia euclídea no es una distancia realista, aproximamos que el camino real es aproximadamente un 150% de la distancia euclídea, ya que no es un camino recto, por lo que en el caso de la distancia, añadimos la penalización*1.5, mientras que para la duración del trayecto, ya que la velocidad media de una persona andando es de 1.4 m/s y asumimos que el trayecto es 1.5 veces la distancia euclídea, añadimos la penalización*1.5/1.4, o sea penalización*~1.07.

```
def get_penalization(old, new):
    old_layer = project.mapLayersByName(old)[0]
    old_features = list(old_layer.getFeatures())
    new_layer = project.mapLayersByName(new)[0]
    new_features = list(new_layer.getFeatures())
    penalizations = {}
    for old_feature in old_features:
        for new_feature in new_features:
            if old_feature["CDTNUCLEO"]==new_feature["CDTNUCLEO"]:
                dist = QgsDistanceArea().measureLine(
                    old_feature.geometry().centroid().asPoint(),
                    new_feature.geometry().centroid().asPoint()
                )
                if dist>0:
                    penalizations[old_feature["CDTNUCLEO"]] = dist
    return penalizations

def apply_penalization(objective,pen_json,):
    penalizations = json_to_dict(pen_json)
    results = json_to_dict(objective)
    for centro_key , penalization in penalizations.items():
        for result_key, result in results[centro_key].items():
            if "transit" in result_key:
                if "dis" in result_key:
                    results[centro_key][result_key]= result +int(penalization*1.5)
                if "dur" in result_key:
                    results[centro_key][result_key]= result +int(penalization*1.07)
    return results
```

Código 7 Métodos `get_penalization()` y `apply_penalization()`

El resultado de estos métodos es un nuevo archivo JSON con un formato mucho más legible que nos permite operar sobre el dataset de forma más sencilla. Para cada núcleo poblacional, podemos ver en la Ilustración 16 que se almacenan todos los datos relacionados con dicho núcleo con la notación

“<dis/dur>_<destino>_<modo>_<hora>”. Los datos de distancia se representan en metros y los de duración en segundos. En caso de los hospitales, se lleva un conteo del número de hospitales de los que se tiene información, por lo que <destino> pasa a ser “hospt<grupo>_<número>”

```
1  "0790101": {
2    "dis_hospt3_1_drive_08:30": 1929,
3    "dur_hospt3_1_drive_08:30": 464,
4    "dis_hospt3_1_drive_11:00": 1929,
5    "dur_hospt3_1_drive_11:00": 476,
6    "dis_hospt3_1_transit_08:30": 2174,
7    "dur_hospt3_1_transit_08:30": 1063,
8    "dis_hospt3_1_transit_11:00": 2213,
9    "dur_hospt3_1_transit_11:00": 1317,
10   "dis_hospt3_2_drive_08:30": 8244,
11   "dur_hospt3_2_drive_08:30": 793,
12   "dis_hospt3_2_drive_11:00": 8244,
13   "dur_hospt3_2_drive_11:00": 835,
14   "dis_hospt3_2_transit_08:30": 7175,
15   "dur_hospt3_2_transit_08:30": 2281,
16   "dis_hospt3_2_transit_11:00": 7175,
17   "dur_hospt3_2_transit_11:00": 2341,
18   "dis_hospt2_1_drive_08:30": 14163,
19   "dur_hospt2_1_drive_08:30": 1467,
20   "dis_hospt2_1_drive_11:00": 14163,
21   "dur_hospt2_1_drive_11:00": 1489,
22   "dis_hospt2_1_transit_08:30": 12782,
23   "dur_hospt2_1_transit_08:30": 3163,
24   "dis_hospt2_1_transit_11:00": 12782,
25   "dur_hospt2_1_transit_11:00": 3223,
26   "dis_juzgados_drive_08:30": 1043,
27   "dur_juzgados_drive_08:30": 253,
28   "dis_juzgados_drive_11:00": 1043,
29   "dur_juzgados_drive_11:00": 281,
30   "dis_juzgados_transit_08:30": 1242,
31   "dur_juzgados_transit_08:30": 702,
32   "dis_juzgados_transit_11:00": 825,
33   "dur_juzgados_transit_11:00": 754,
34   "dis_bomberos_drive_08:30": 1633,
35   "dur_bomberos_drive_08:30": 403,
36   "dis_bomberos_drive_11:00": 1633,
37   "dur_bomberos_drive_11:00": 455,
38   "dis_bomberos_transit_08:30": 1827,
39   "dur_bomberos_transit_08:30": 1052,
40   "dis_bomberos_transit_11:00": 1652,
41   "dur_bomberos_transit_11:00": 1166,
42   "dis_csmental_drive_08:30": 1582,
43   "dur_csmental_drive_08:30": 402,
44   "dis_csmental_drive_11:00": 1582,
45   "dur_csmental_drive_11:00": 458,
46   "dis_csmental_transit_08:30": 1293,
```

Ilustración 16 Ejemplo de JSON despues de penalizaciones

4.6. Unión con el dataset en formato Geopackage

Una vez tenemos todos los datos en formato JSON está todo preparado para el siguiente paso: unir los resultados obtenidos con las capas de proyecto QGIS.

Este proceso es quizás el más largo dentro del procesamiento de datos y las funciones implementadas son demasiado largas como para ser presentadas, pero esta es una

descripción de los pasos a seguir, además de proveer un vistazo de los resultados obtenidos.

Capa de núcleos poblacionales

Primero fusionamos los resultados obtenidos con la capa de centroides de los núcleos poblacionales. Debido a que cada núcleo poblacional tiene un número distinto de hospitales cercanos:

- El máximo de hospitales de grupo 2 cercanos es 4, por lo que cada núcleo poblacional tendrá columnas para información de 4 hospitales de grupo 2, si un núcleo en específico tiene menos, las columnas toman valor nulo.
- El máximo de hospitales de grupo 3 cercanos es 3, se toma la misma medida que en el punto anterior.

Además de estos datos añadimos varios atributos extra:

- Media de distancia y duración a cada servicio público (10 columnas)
- Media de las medias de distancia y duración a cada servicio público (2 columnas)

Por último, para facilitar el paso posterior añadimos 12 nuevas columnas a partir de multiplicar las 12 columnas anteriores por la población de cada núcleo poblacional. Esto nos permitirá en el paso posterior calcular una media ponderada que tenga en cuenta la población de cada núcleo.

Antes de introducir las atributos añadidos, debemos explicar el significado de ciertas variables:

- `<tipo>` toma los valores:
 - `“dis”`: Si informa sobre la distancia (en metros)
 - `“dur”`: Si informa sobre la duración de viaje (en segundos)
- `<destino>` toma los valores:
 - `“bomberos”`:
 - `“csmental”`:
 - `“juzgados”`:
 - `hospt<3|2>_<n>`: Como se explica al principio de este punto. Puede haber datos de varios hospitales de cada tipo. En caso de ser el hospital de grupo 2 número 3 sería `“hospt2_3”`
- `<modo>` toma los valores:
 - `“drive”`: Para trayectos en carretera
 - `“transit”`: Para trayectos en transporte público
- `<hora>` toma los valores:
 - `“08:30”` para trayectos en hora punta
 - `“11:00”` para trayectos con tráfico relajado

Una vez explicado esto, aquí podemos ver un desglose de los atributos añadidos al dataset de núcleos poblacionales a partir de los resultados obtenidos de Routes API:

Atributo	Descripción
<tipo>_<destino>_<modo>_<hora>	Esta descripción abarca un total de 80 columnas con los datos originales obtenidos de Routes API, algunos con modificaciones debido a las penalizaciones aplicadas en el punto 4.5 Depuración de datos
mean_<tipo>_<destino>_<modo>	20 columnas. La media de distancia o duración para cada tipo de servicio público en coche o transporte público
mean_<tipo>_<destino>_all	10 columnas. La media de distancia o duración por cada tipo de servicio público
mean_<tipo>_<modo>	4 columnas: • mean_dis_drive: Media de todas las distancias en coche • mean_dis_transit: Media de todas las distancias en transporte público • mean_dur_drive: media de todas las duraciones en coche. • Mean_dur_tranist: Media de todas las duraciones en transporte público
mean_<tipo>_all	2 columnas: • mean_dis_all: Media de todas las distancias saliendo del núcleo • mean_dur_all: Media de todas las duraciones de trayecto saliendo del núcleo
weigh_mean....	36 columnas, cada una de las columnas que comienzan por “mean” multiplicadas por la población del núcleo. Útil para el cálculo de la media ponderada en la capa de municipios

Esto significa que se añaden 152 nuevos atributos al dataset de núcleos poblacionales que, junto a los 12 anteriores, nos deja con un dataset con información sobre 468 núcleos poblacionales y 164 atributos.

Capa de municipios

A la hora de crear la capa de núcleos poblacionales nos aseguramos de crear atributos con datos útiles para calcular la media ponderada en esta capa. Estos valores los llamaremos valores ponderados y sumaremos todos los valores ponderados de cada núcleo dentro de cada municipio y lo dividiremos por la población total de cada municipio.

$$valor_{pond} = valor * poblacion,$$

$$media_{pond} = \frac{\sum valor_{pond}}{\sum poblacion}$$

A partir de esta operación obtenemos 36 atributos con las medias ponderadas de atributos clave de los núcleos poblacionales de cada municipio, además de tener en cuenta la población de cada núcleo para obtener resultados más precisos.

Como resultado de esta operación y unir dichos atributos con la capa, damos por terminado el procesamiento de datos

4.7. Desarrollo de un índice de accesibilidad

Desde un principio, la idea del proyecto, más que crear un índice estandarizado para el cálculo de la disponibilidad y accesibilidad de servicios públicos, es la obtención de una gran cantidad de atributos que ya de por sí aporten la suficiente información sobre estos factores.

Indicadores directos obtenidos:

Para cada dataset (núcleos poblacionales y municipios) destacan los siguientes indicadores de accesibilidad:

- **mean_dis_<destino>_<modo>**: Distancia media en metros a cada tipo de servicio público por modo de transporte
- **mean_dur_<destino>_<modo>**: Tiempo medio en segundos para acceder a cada tipo de servicio público por modo de transporte
- **mean_dis_all** y **mean_dur_all**: Indicadores globales que promedian el acceso a todos los servicios

Interpretación de los indicadores:

- Valores menores indican mayor accesibilidad
- Los datos permiten comparaciones directas entre núcleos/municipios
- Se pueden establecer umbrales críticos (ej: > 30 min para hospitales)
- La diferenciación por modo de transporte refleja distintas realidades socioeconómicas

Ventajas de este enfoque:

- **Transparencia**: Los valores son directamente interpretables sin transformaciones complejas
- **Flexibilidad**: Permite a usuarios finales aplicar sus propios criterios de evaluación
- **Granularidad**: Mantiene la información específica por servicio y modo de transporte
- **Comparabilidad**: Facilita análisis territoriales y identificación de desigualdades

Este enfoque proporciona una base sólida para análisis de equidad territorial sin imponer una metodología específica de agregación de indicadores.

5. DESCRIPCIÓN DEL DATASET OBTENIDO

5.1. Modelo de datos

Este proyecto trabaja con un modelo de datos geoespaciales que organiza la información sobre accesibilidad territorial de forma práctica y reutilizable. En lugar de crear una ontología compleja, se ha optado por una estructura de datos clara que cualquier herramienta GIS puede procesar.

5.1.1. Datasets generados

El análisis produce dos conjuntos de datos principales en formato GeoPackage:

Centroides de núcleos poblacionales

Este dataset contiene puntos que representan los centroides de cada núcleo poblacional de la Comunidad de Madrid. Cada punto incluye:

- **Localización:** Coordenadas en sistema WGS84 (EPSG:4326)
- **Identificación:** Códigos únicos para cada núcleo
- **Métricas de acceso:** Distancias y tiempos medios a diferentes servicios
- **Variaciones por transporte:** Datos separados para desplazamiento a pie y en coche

Datos municipales agregados

Los límites municipales con información resumida que incluye:

- **Geometrías:** Polígonos oficiales de cada municipio
- **Códigos administrativos:** Identificadores LAU oficiales
- **Estadísticas agregadas:** Promedios calculados a partir de los núcleos contenidos
- **Indicadores síntesis:** Valores que resumen la accesibilidad general

5.1.2. Estructura de los campos de datos

Los campos siguen un patrón sistemático para facilitar su interpretación:

Distancias medias: Los campos que empiezan por `mean_dis_` contienen distancias en metros. Por ejemplo:

- `mean_dis_hospitals_walking`: distancia media a hospitales caminando

- `mean_dis_schools_driving`: distancia media a colegios en coche

Tiempos medios: Los campos `mean_dur_` guardan tiempos en segundos:

- `mean_dur_hospitals_walking`: tiempo medio para llegar a hospitales a pie
- `mean_dur_schools_driving`: tiempo medio para llegar a colegios en coche

Consideraciones temporales: Algunos campos incluyen información sobre la hora del día:

- Sufijo `_peak`: condiciones de hora punta
- Sufijo `_normal`: condiciones de tráfico normal

5.1.3. Tipos de servicios analizados

Los datos cubren varios tipos de servicios esenciales:

- **Sanidad:** Hospitales, centros de salud, farmacias
- **Educación:** Colegios públicos y privados, institutos
- **Transporte público:** Estaciones de tren, metro, paradas de bus
- **Administración:** Ayuntamientos, oficinas de la Seguridad Social
- **Comercio básico:** Supermercados y centros comerciales

5.1.4. Modos de transporte considerados

A pie (walking): Representa la accesibilidad más básica, importante para personas sin vehículo o con movilidad reducida. Los cálculos consideran la topografía y la infraestructura peatonal.

En vehículo privado (driving): Refleja las condiciones de acceso motorizado, incluyendo el estado del tráfico y las restricciones de circulación. Se diferencia entre hora punta y hora normal.

5.1.5. Relaciones entre los datos

Aunque no hay una ontología formal, los datos mantienen conexiones lógicas:

Relación espacial: Los núcleos poblacionales pertenecen a municipios específicos, lo que permite agregar datos desde la escala local hasta la municipal.

Relación temporal: Las mediciones reflejan diferentes momentos del día, permitiendo analizar cómo varía la accesibilidad según la hora.

Relación funcional: Los indicadores agregados (`mean_dis_all`, `mean_dur_all`) resumen la accesibilidad general combinando todos los servicios.

5.1.6. Calidad de los datos

Cobertura territorial: Se incluyen todos los municipios de la Comunidad de Madrid, asegurando una visión completa del territorio.

Consistencia: Todos los datasets usan las mismas unidades (metros para distancia, segundos para tiempo) y el mismo sistema de coordenadas.

Actualización: Los datos reflejan la situación de la infraestructura de servicios durante el período de análisis (julio-agosto 2025).

5.2. Documentación con estándares web

Para que los datos sean fáciles de encontrar y reutilizar, se ha usado el estándar DCAT (Data Catalog Vocabulary) recomendado por la W3C. Este vocabulario permite describir datasets de forma que sean comprensibles tanto para personas como para sistemas automáticos.

5.2.1. Información del catálogo

El catálogo principal incluye:

- **Identificación:** URL del repositorio donde están los datos
- **Descripción:** Explicación clara de qué contienen los datos
- **Responsable:** Información sobre quién ha creado los datos
- **Licencia:** Creative Commons BY 4.0 que permite uso libre con atribución
- **Idioma:** Español
- **Fechas:** Cuándo se crearon y actualizaron por última vez

5.2.2. Metadatos específicos por dataset

Dataset de núcleos poblacionales

Campo	Valor
Título	Núcleos poblacionales
Descripción	Centroides con métricas de accesibilidad
Formato	GeoPackage (GPKG)
Tamaño	468 centroides
Cobertura	Comunidad de Madrid, España
Período	Julio-Agosto 2025
URL descarga	GitHub Releases

Dataset de municipios

Campo	Valor
Título	Municipios
Descripción	Límites municipales con datos agregados
Formato	GeoPackage (GPKG)
Tamaño	179 municipios
Cobertura	Comunidad de Madrid, España
Período	Julio-Agosto 2025
URL descarga	GitHub Releases

5.2.3. Ventajas del enfoque DCAT

Usar este estándar aporta varios beneficios:

Facilita el descubrimiento: Los buscadores especializados en datos utilizan el formato DCAT indexar y encontrar datasets de forma automática.

Mejora la reutilización: Otros investigadores tienen toda la información necesaria para usar los datos correctamente.

Garantiza la sostenibilidad: Los metadatos permiten que los datos sigan siendo útiles en el futuro, aunque cambien las herramientas de software.

Cumple normativas: Se alinea con las directivas europeas sobre datos abiertos y transparencia.

5.2.4. Acceso a los datos

Los datos están disponibles públicamente a través de:

- **Repositorio GitHub:** Código fuente y documentación
- **GitHub Releases:** Archivos GPKG listos para descargar
- **Documentación:** README con instrucciones de uso
- **Metadatos:** Archivo JSON-LD con toda la información descriptiva

Esta aproximación hace que el proyecto sea un recurso de datos abiertos que puede servir para futuras investigaciones, planificación territorial o desarrollo de aplicaciones relacionadas con la accesibilidad urbana.

6. ESPECIFICACIÓN DE REQUISITOS DE LA APLICACIÓN GIS PARA VISUALIZAR LOS DATOS

En este punto definimos los requisitos que marcan las directrices en el desarrollo de un visor web sencillo de los datasets creados. Definiremos los requisitos, tanto funcionales como no funcionales en un formato estandarizado.

Campo	Descripción
ID de requisito	RF<N>. Siendo N el número del requisito funcional.
Título	Palabra o frase breve describiendo el requisito
Descripción	Descripción extensa de los requisitos

Tabla 1 Plantilla de descripción de requisitos

A continuación, en el punto 6.1.se profundiza en los requisitos que han guiado el desarrollo de la aplicación web diseñada para la visualización de datos de atractividad territorial. Estos requisitos funcionales han sido fundamentales para definir el alcance del proyecto, garantizar su operatividad y asegurar una buena experiencia de usuario.

El formato empleado para describir los requisitos se resume en la Tabla 1. Este formato permite organizar de forma clara y estructurada cada uno de los elementos que definen los objetivos funcionales del sistema.

6.1. Especificación de Requisitos

Campo	Descripción
ID de requisito	RF1
Título	Carga de los datos
Descripción	El visor web carga los datasets de municipios y núcleos poblacionales almacenados localmente. Entrada: Archivos en formato GeoPackage o GeoJSON con los datos de accesibilidad, tanto para núcleos poblacionales como para municipios Proceso: Al ejecutar el script de ejecución del visor, carga los archivos en memoria. Salida: 2 capas con las que se puede interactuar en el visor web

Campo	Descripción
ID de requisito	RF2
Título	Selección de capa
Descripción	El visor permite mostrar o ocultar las capas de nuestra elección Entrada: Activación o desactivación de un interruptor en la página Proceso: Al activar el interruptor, la capa se vuelve visible al usuario y al desactivarla se oculta Salida: Las capas elegidas por el usuario

Campo	Descripción
ID de requisito	RF3
Título	Selección de atributo de ordenación
Descripción	El visor permite elegir, para cada capa, un atributo y se presenta un Entrada: Selección de un atributo en un dropdown Proceso: El visor web normaliza los valores de el atributo y le aplica un gradiente verde-rojo a las capas afectadas, dándole un color rojo cuando el valor es alto y verde cuando es bajo Salida: Representación visual sencilla de los atributos de las capas

Campo	Descripción
ID de requisito	RF4
Título	Visualización de atributos
Descripción	El visor permite seleccionar un punto en el mapa, bien sea un núcleo o un municipio y muestra los atributos de la instancia de datos seleccionada. Entrada: Click en un punto específico del mapa Proceso: El visor web detecta el punto pulsado y carga los atributos de dicho núcleo/municipio Salida: Listado en un lado de la pantalla con los atributos y su valor

Campo	Descripción
ID de requisito	RF5
Título	Selección de distintas capas en el background
Descripción	El visor permite seleccionar varias capas para el background, permitiendo tener un punto de referencia y no un fondo blanco. Las capas son: • OSM (OpenStreetMap) • Satélite • Topográfico Entrada: Click en un menú dropdown con las distintas capas Proceso: El visor web carga la capa seleccionada en el background sin tapar las ya existentes Salida: Un background acorde a la selección detrás de las capas del dataset

Campo	Descripción
ID de requisito	RF6
Título	Caja con coordenadas del ratón en tiempo real

Descripción	El visor aporta una pequeña caja en una esquina del visor con las coordenadas en tiempo real del ratón Entrada: Movimiento del ratón Proceso: En cada movimiento del ratón el visor actualiza las coordenadas mostradas en una pequeña caja del visor web Salida: Una pequeña caja informativa con la latitud y longitud en la que se encuentra el ratón
-------------	---

Campo	Descripción
ID de requisito	RF7
Título	Control de opacidad de capas
Descripción	El visor aporta un pequeño slider que permite controlar la opacidad de las capas, en caso de querer ver varias capas a la vez Entrada: Movimiento del slider Proceso: A medida que se mueve el slider el visor va cambiando la opacidad de la capa seleccionada Salida: Cambio en la opacidad de la capa deseada

7. DISEÑO DEL VISOR WEB

La visualización de datos territoriales de accesibilidad requiere herramientas que permitan explorar los resultados de forma intuitiva acompañados de una visualización del territorio desrito. Para abordar esta necesidad, se ha desarrollado un visor web GIS que facilita el análisis interactivo de las métricas de accesibilidad calculadas para la Comunidad de Madrid.

El desarrollo del visor se fundamenta en tecnologías web que garantizan compatibilidad, rendimiento y facilidad de uso. La aplicación permite visualizar simultáneamente datos municipales y de núcleos poblacionales, ofreciendo diferentes niveles de detalle según las necesidades del análisis.

7.1. Tecnología utilizada

La elección de tecnologías para el desarrollo del visor se basó en criterios de simplicidad, rendimiento y compatibilidad con estándares web actuales.

7.1.1. Backend: Flask como servidor ligero

Se optó por **Flask** como framework para el servidor backend debido a su simplicidad y flexibilidad para proyectos de tamaño medio. Flask permite crear APIs REST de forma rápida sin la complejidad de frameworks más pesados.

Ventajas de Flask para este proyecto:

- Configuración mínima requerida
- Facilidad para crear APIs JSON
- Fácil integración con librerías geoespaciales como GeoPandas
- Posibilidad de optimización

El servidor implementa una estrategia de **precarga en memoria** que mejora significativamente el rendimiento. Los archivos GeoJSON se cargan una sola vez al iniciar el servidor, eliminando la necesidad de leer archivos del disco en cada petición.

7.1.2. Frontend: Tecnologías web estándar

Para el frontend se utilizaron tecnologías web nativas sin frameworks complejos:

- **HTML5:** Estructura semántica del documento y contenedores principales
- **CSS3:** Estilos responsivos y diseño de la interfaz de usuario
- **JavaScript ES6+:** Lógica de interacción, gestión de capas y comunicación con la API

Esta aproximación garantiza compatibilidad con todos los navegadores modernos y facilita el mantenimiento del código.

7.1.3. Cartografía: OpenLayers para visualización geoespacial

La visualización cartográfica se implementó con **OpenLayers**, una librería JavaScript especializada en mapas web interactivos.

Características relevantes de OpenLayers:

- Soporte nativo para múltiples formatos (GeoJSON, WMS, WFS)
- Renderizado vectorial eficiente en el navegador
- Controles de navegación y zoom integrados
- Gestión avanzada de capas y estilos
- Compatibilidad con proyecciones cartográficas estándar

7.2. Arquitectura del sistema

7.2.1. Arquitectura general

El visor implementa una arquitectura cliente-servidor simple pero efectiva:

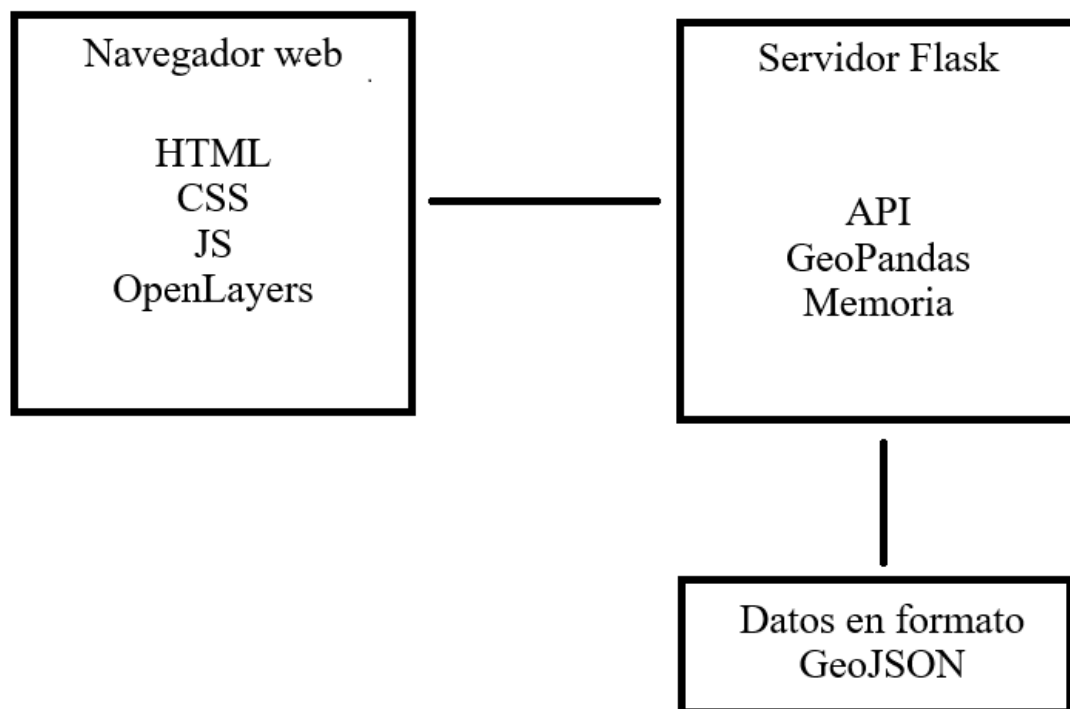


Ilustración 17 Simplificación de la arquitectura del visor web

7.2.2. Flujo de datos

1. **Inicio del servidor:** Flask precarga los archivos GeoJSON en memoria usando GeoPandas
2. **Petición inicial:** El navegador solicita la página HTML principal

3. **Carga de datos:** JavaScript solicita las capas geográficas a través de la API REST
4. **Renderizado:** OpenLayers procesa los datos GeoJSON y los visualiza en el mapa
5. **Interacción:** Los cambios en la interfaz activan nuevas consultas o actualizaciones visuales

7.2.3. API REST

El backend expone los siguientes endpoints:

Endpoint	Método	Descripción
/api/municipios	GET	Geometrías completas de municipios
/api/centroides	GET	Puntos de núcleos poblacionales
/api/municipios_simple	GET	Geometrías simplificadas para mejor rendimiento
/api/health	GET	Verificación del estado del servidor

Todos los endpoints incluyen headers de caché HTTP para optimizar la carga de datos.

7.3. Diseño de la interfaz de usuario

7.3.1. Principios de diseño

El diseño de la interfaz se guió por principios de usabilidad y accesibilidad:

Simplicidad: Interfaz limpia sin elementos innecesarios que distraigan del análisis

Consistencia: Elementos de control uniformes en toda la aplicación

Retroalimentación: Respuestas visuales inmediatas a las acciones del usuario

Flexibilidad: Capacidad de personalizar la visualización según necesidades específicas

7.3.2. Organización espacial

La interfaz se estructura en dos áreas principales:

Panel de control lateral (izquierda)

- **Control de capas:** Activación/desactivación de municipios y núcleos
- **Ajuste de opacidad:** Sliders para controlar transparencia de capas
- **Selección de atributos:** Dropdown para elegir qué variable visualizar
- **Estadísticas:** Información numérica sobre los datos cargados
- **Panel informativo:** Detalles contextuales sobre elementos seleccionados

Área de mapa principal (derecha)

- **Visualización cartográfica:** Mapa interactivo con las capas de datos
- **Controles de navegación:** Zoom, centrado y herramientas de exploración
- **Indicadores de posición:** Coordenadas del cursor en tiempo real
- **Escala gráfica:** Referencia visual de distancias

7.3.3. Esquema de colores y simbolización

Paleta cromática: Se utiliza una escala de colores verde-amarillo-rojo que facilita la interpretación intuitiva:

- Verde: Valores bajos (mejor accesibilidad)
- Amarillo: Valores medios
- Rojo: Valores altos (menor accesibilidad)

Simbolización por capas:

- **Municipios:** Polígonos con relleno colorimétrico y contorno negro
- **Núcleos poblacionales:** Círculos proporcionales con contorno blanco
- **Transparencia:** Ajustable independientemente para cada capa

7.3.4. Mockup final del visor

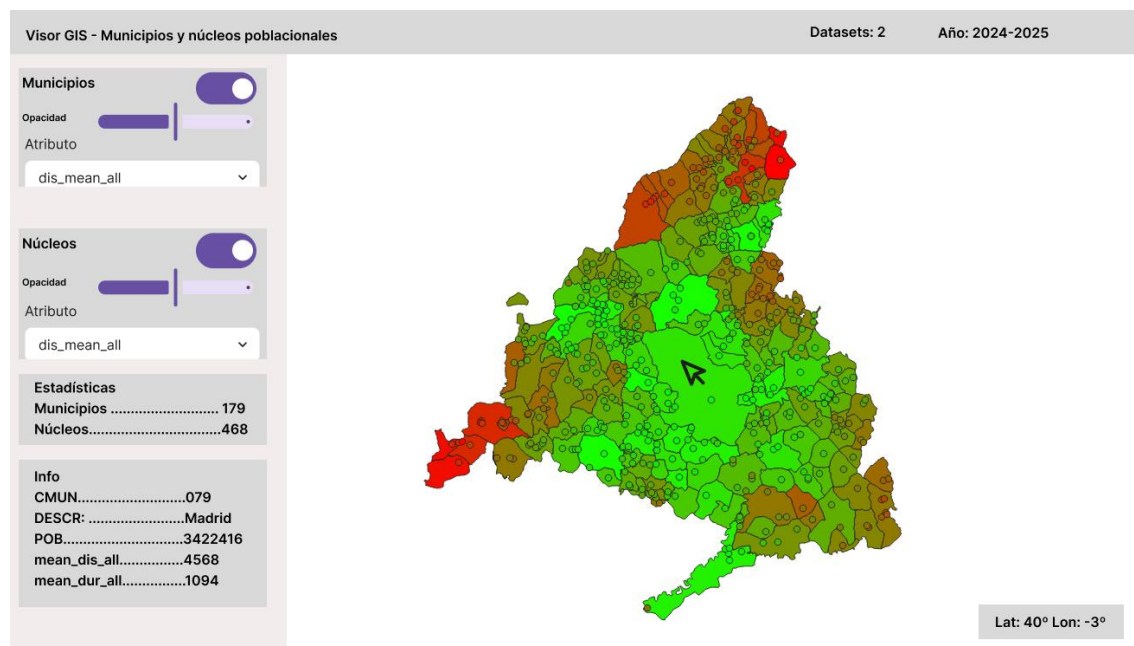


Ilustración 18 Mockup del visor web deseado hecho en FIGMA

7.4. Implementación técnica

7.4.1. Estructura del proyecto

/Datos nuevos/

```
| municipios.geojson # Datos municipales
| centroides.geojson # Datos de núcleos
/visor/

| server3.py          # Servidor Flask principal
| index.html          # Página web principal
```

7.4.2. Optimizaciones implementadas

Carga eficiente de datos:

- Precarga de GeoJSON en memoria del servidor
- Simplificación geométrica opcional para mejorar rendimiento
- Headers de caché HTTP para evitar transferencias innecesarias

Renderizado optimizado:

- Uso de geometrías simplificadas para vistas de datasets de gran escala
- Actualización selectiva de capas según cambios de usuario
- Debouncing en controles deslizantes para evitar actualizaciones excesivas

Gestión de memoria:

- Liberación automática de recursos cartográficos no utilizados
- Reutilización de objetos geográficos cuando es posible

7.4.3. Funcionalidades principales

Visualización multi-capa:

- Activación independiente de capas de municipios y núcleos
- Control granular de transparencia para cada capa
- Superposición configurable de elementos cartográficos

Exploración de atributos:

- Selección dinámica de variables a visualizar
- Actualización automática de leyenda y colores
- Información contextual sobre rangos de valores

Interacción espacial:

- Navegación libre por zoom y paneo
- Información emergente al hacer clic en elementos
- Indicadores de posición geográfica en tiempo real

Estadísticas en tiempo real:

- Conteo automático de elementos visibles
- Cálculo de estadísticas básicas de la variable seleccionada
- Información de contexto sobre el elemento bajo el cursor

7.5. Experiencia de usuario

7.5.1. Flujo de trabajo típico

1. **Acceso inicial:** El usuario accede a la aplicación y observa el mapa general de la Comunidad de Madrid
2. **Selección de datos:** Activa las capas de interés (municipios y/o núcleos poblacionales)
3. **Exploración temática:** Selecciona un atributo específico para visualizar (ej: mean_dis_all)
4. **Ajuste visual:** Modifica la opacidad de las capas para optimizar la visualización
5. **Análisis espacial:** Navega por el territorio identificando patrones y áreas de interés
6. **Consulta específica:** Hace clic en elementos específicos para obtener información precisa

7.5.2. Beneficios para el análisis territorial

Análisis comparativo: La visualización simultánea permite comparar fácilmente la accesibilidad entre municipios

Detección de patrones: Los gradientes de color revelan patrones espaciales no evidentes en datos tabulares

Análisis multi-escala: La combinación de datos municipales y de núcleos ofrece perspectivas complementarias

Exploración interactiva: La navegación libre permite descubrir relaciones espaciales inesperadas

Esta implementación proporciona una herramienta práctica y eficiente para el análisis de accesibilidad territorial, facilitando la comprensión de los datos generados y apoyando la toma de decisiones basada en evidencia espacial.

8. RESULTADOS

Resumen de los resultados:

- Distancias muy largas en municipios de la periferia
- Concentración de servicios en el centro
- Más.

9. CONCLUSIONES Y FUTUROS TRABAJOS

Referencias

- [1 Instituto Nacional de Estadística, «Núcleo de población,» [En línea]. Available:
] <https://www.ine.es/DEFIne/es/concepto.htm?c=4930>. [Último acceso: 2025].
- [2 Instituto Nacional de Estadística, «Diseminado,» [En línea]. Available:
] <https://www.ine.es/DEFIne/concepto.htm?c=5188>.
- [3 I. G. Nacional, «Información de datos geográficos - Instituto Geográfico Nacional,»
] 2011. [En línea]. Available: <https://www.ign.es/web/ane-datos-geograficos/-/datos-geograficos/datosPoblacion?tipoBusqueda=CCAA>. [Último acceso: 2025].
- [4 I. N. d. Estadística, «Madrid: Población por municipios y sexo. (2881),» 2024. [En
] línea]. Available: https://ine.es/jaxiT3/Datos.htm?t=2881#_tabs-tabla. [Último
acceso: 2025].
- [5 A. Moreno-Jiménez y A. Roca, «Accesibilidad de los jóvenes a la red de transporte
] público en Alcobendas (Madrid) por motivo de ocio nocturno,» *ResearchGate*,
2012.
- [6 T. Neutens, «Accessibility, equity and health care: review and research directions
] for transport geographers,» *Journal of Transport Geography*, vol. 43, pp. 14-27,
2015.
- [7 Pechansky, R. D. y J. W. Thomas, «The Concept of Access: Definition and
] Relationship to Consumer Satisfaction,» *Medical Care*, vol. 19, nº 2, pp. 127-140,
1981.
- [8 Instituto de Estadística de la Comunidad de Madrid, «Nomecalles,» [En línea].
] Available: https://gestion.comunidad.madrid/nomecalles_web/#/inicio.
- [9 «Datos Abiertos del Consorcio Regional de Transportes de Madrid,» 2025. [En
] línea]. Available: <https://data-crtm.opendata.arcgis.com/search>.
- [1 K. Mirwaldt, I. McMaster y J. Bachtler, «Reconsidering Cohesion Policy : the
0] Contested debate on Territorial Cohesion,» Preprint / Working Paper. University of
Strathclyde, Glasgow, 2009.
- [1 M. Taleai, R. Sliuzas y J. Flacke, «An integrated framework to evaluate the equity
1] of urban public facilities using spatial multi-criteria analysis,» *Cities*, vol. 40, pp.
56-69, 2014.
- [1 I. Y. J. J. L. y E. H. C. , «Spatial justice in public open space planning: Accessibility
2] and inclusivity,» *Habitat International*, vol. 97, 2020.

- [1 L. C. e. al., «Access to an Availability of Exercise Facilities in Madrid: An Equity
3] Perspective,» *International Journal of Health Geographics*, vol. 18, nº 1, 2019.
- [1 D. O. d. l. U. Europea, «Directiva (UE) 2019/1024 del Parlamento Europeo y del
4] Consejo, de 20 de junio de 2019, relativa a los datos abiertos y la reutilización de la
información del sector público.,» 2019.
- [1 B. O. d. Estado, «Ley 37/2007, de 16 de noviembre, sobre reutilización de la
5] información del sector público.,» 2007.
- [1 EUR-Lex, «Consolidated version of the Treaty on the Functioning of the European
6] Union,» 2012.
- [1 D. Gutiérrez, «Recorrer 60 Kilómetros Para Llevar a Tu Hijo al Pediatra: Así Se
7] Vive En Algunos Pueblos de Madrid.,» *El País*, pp. elpais.com/espana/madrid/2024-12-07/recorrer-60-kilometros-para-llevar-a-tu-hijo-al-pediatra-asi-se-vive-en-algunos-pueblos-de-madrid.html, 7 12 2024.
- [1 B. Olaizola, «Una biblioteca por cada 101.000 madrileños: la capital tiene poas
8] instalaciones públicas y mal repartidas,» *El País*, pp.
<https://elpais.com/espana/madrid/2023-09-03/radiografia-de-instalaciones-publicas-en-madrid-insuficientes-y-mal-repartidas.html>, 3 9 2023.
- [1 Google Maps Platform Documentation | Routes API | Google for Developers, [En
9] línea]. Available: <https://developers.google.com/maps/documentation/routes>.
[Último acceso: 2025].
- [2 «Centro de Descargas del CNIG,» [En línea]. Available:
0] <https://centrodedescargas.cnig.es/CentroDescargas/home>.
- [2 Instituto Nacional de Estadística, «Nomenclátor. Población por unidad poblacional,»
1] [En línea]. Available: <https://www.ine.es/nomen2/>.
- [2 E. R. a. o. Frank Warmerdam, «OGR SQL dialect,» [En línea]. Available:
2] https://gdal.org/en/stable/user/ogr_sql_dialect.html. [Último acceso: 2025].
- [2 M. Koenigbauer, «QGIS Plugins - ProcessX,» [En línea]. Available:
3] <https://plugins.qgis.org/plugins/ProcessX>.
- [2 Comunidad de Madrid, «Hospitales de la red del Servicio Madrileño de Salud,» [En
4] línea]. Available: <https://www.comunidad.madrid/servicios/salud/hospitales-red-servicio-madrileno-salud>.
- [2 Boletín Oficial del Estado, «Artículo 32 Ley Orgánica del Poder judicial,» [En
5] línea]. Available: <https://www.iberley.es/legislacion/articulo-32-ley-organica-poder-judicial>.

- [2] Ministerio de justicia de España, «Nomenclátor de Partidos Judiciales de España,» 6] [En línea]. Available: https://www.mjusticia.gob.es/es/JusticiaEspana/OrganizacionJusticia/Documents/011223_Nomenc%C3%A1tor%20de%20Partidos%20Judiciales.pdf.
- [2] Procuradora Asociados, «Buscador de PARTIDOS JUDICIALES EN Madrid,» [En 7] línea]. Available: <https://procuradoresasociadosmadrid.es/partidos-judiciales-madrid/>.
- [2] QGIS.org, «Spatial without Compromise · QGIS Website,» QGIS Association, 8] 2025. [En línea]. Available: <http://www.qgis.org>.